In [27]:
```
import pandas as pd
from datetime import datetime
```

Cargar Datos

In [28]:
```
equipos  = pd.read_csv(r"C:\Users\Sebastian.garzon.LAPTOP-G9J39UDS\Documents\Cosas\
X  = pd.read_csv(r"C:\Users\Sebastian.garzon.LAPTOP-G9J39UDS\Documents\Cosas\Cosas\
Y  = pd.read_csv(r"C:\Users\Sebastian.garzon.LAPTOP-G9J39UDS\Documents\Cosas\Cosas\
Z  = pd.read_csv(r"C:\Users\Sebastian.garzon.LAPTOP-G9J39UDS\Documents\Cosas\Cosas\
```

Compocisión de los datos

In [29]:
```
print(equipos.shape)
print(X .shape)
print(Y.shape)
print(Z.shape)
```

```
(3530, 6)
(9144, 2)
(4485, 2)
(3565, 2)
```

In [30]:
```
X
```

Out[30]:

	Date	Price
0	2024-04-04	89.18
1	2024-04-03	89.35
2	2024-04-02	88.92
3	2024-04-01	87.42
4	2024-03-28	87.48
...	...	...
9139	1988-07-01	14.60
9140	1988-06-30	14.85
9141	1988-06-29	15.47
9142	1988-06-28	15.27
9143	1988-06-27	15.10

9144 rows × 2 columns

Analisis de las fechas

In [31]:
```
print(equipos["Date"].max(), equipos["Date"].min())
print(X["Date"].max(), X["Date"].min())
print(Y["Date"].max(), Y["Date"].min())
print(Z["Date"].max(), Z["Date"].min())
```

```
2023-08-31 2010-01-04
2024-04-04 1988-06-27
9/9/2022 1/1/2007
2023-08-31 2010-01-01
```

Converitr las fechas en formato Date

In [32]:
```
equipos["Date"] = pd.to_datetime(equipos["Date"], format= "%Y-%m-%d" ).dt.date
X["Date"] = pd.to_datetime(X["Date"], format= "%Y-%m-%d").dt.date
Y["Date"] = pd.to_datetime(Y["Date"], format= "%d/%m/%Y").dt.date
Z["Date"] = pd.to_datetime(Z["Date"], format= "%Y-%m-%d" ).dt.date
```

In [33]:
```
print(equipos["Date"].max(), equipos["Date"].min())
print(X["Date"].max(), X["Date"].min())
print(Y["Date"].max(), Y["Date"].min())
print(Z["Date"].max(), Z["Date"].min())
```

```
2023-08-31 2010-01-04
2024-04-04 1988-06-27
2023-09-12 2006-07-11
2023-08-31 2010-01-01
```

In [34]:
```
print(equipos.columns)
print(X.columns)
print(Y.columns)
print(Z.columns)
```

```
Index(['Date', 'Price_X', 'Price_Y', 'Price_Z', 'Price_Equipo1',
       'Price_Equipo2'],
      dtype='object')
Index(['Date', 'Price'], dtype='object')
Index(['Date', 'Price'], dtype='object')
Index(['Price', 'Date'], dtype='object')
```

Limpieza del texto

In [35]:
```
Y["Price"] = Y["Price"].str.replace(",", ".").astype(float)
```

In [36]:
```
equipos
```

Out[36]:

	Date	Price_X	Price_Y	Price_Z	Price_Equipo1	Price_Equipo2
0	2010-01-04	80.12	527.50	2225.25	434.73	931.73
1	2010-01-05	80.59	527.50	2246.50	449.97	968.56
2	2010-01-06	81.89	527.50	2302.50	444.48	960.51
3	2010-01-07	81.51	527.50	2306.50	440.90	960.14
4	2010-01-08	81.37	552.50	2261.25	448.82	949.55
...	...	...	...	...	...	...
3525	2023-08-25	84.48	554.00	2123.50	450.36	899.65
3526	2023-08-28	84.42	549.00	2116.25	469.03	933.20
3527	2023-08-29	85.49	545.33	2116.25	460.42	901.79
3528	2023-08-30	85.86	543.67	2140.25	464.80	945.40
3529	2023-08-31	86.86	545.00	2165.25	451.73	955.35

3530 rows × 6 columns

In [37]:
```
X
```

Out[37]:

	Date	Price
0	2024-04-04	89.18
1	2024-04-03	89.35
2	2024-04-02	88.92
3	2024-04-01	87.42
4	2024-03-28	87.48
...	...	...
9139	1988-07-01	14.60
9140	1988-06-30	14.85
9141	1988-06-29	15.47
9142	1988-06-28	15.27
9143	1988-06-27	15.10

9144 rows × 2 columns

In [38]:
```
Y
```

Out[38]:

	Date	Price
0	2023-09-12	547.33
1	2023-09-11	546.00
2	2023-09-08	545.00
3	2023-09-07	550.00
4	2023-09-06	552.50
...	...	...
4480	2006-07-17	555.00
4481	2006-07-14	555.00
4482	2006-07-13	555.00
4483	2006-07-12	555.00
4484	2006-07-11	555.00

4485 rows × 2 columns

In [39]:
```
Z
```

Out[39]:

	Price	Date
0	2225.25	2010-01-01
1	2225.25	2010-01-04
2	2246.50	2010-01-05
3	2302.50	2010-01-06
4	2306.50	2010-01-07
...	...	...
3560	2123.50	2023-08-25
3561	2116.25	2023-08-28
3562	2116.25	2023-08-29
3563	2140.25	2023-08-30
3564	2165.25	2023-08-31

3565 rows × 2 columns

Join de las bases para corroborar los datos

In [40]:
```
df_merge = equipos.merge(X.rename(columns={"Price": "Price_X2"}), on= "Date", how =
```

In [41]:
```
df_merge
```

Out[41]:

	Date	Price_X	Price_Y	Price_Z	Price_Equipo1	Price_Equipo2	Price_X2	Price_Y2	P
0	2010-01-04	80.12	527.50	2225.25	434.73	931.73	80.12	527.50	
1	2010-01-05	80.59	527.50	2246.50	449.97	968.56	80.59	527.50	
2	2010-01-06	81.89	527.50	2302.50	444.48	960.51	81.89	527.50	
3	2010-01-07	81.51	527.50	2306.50	440.90	960.14	81.51	527.50	
4	2010-01-08	81.37	552.50	2261.25	448.82	949.55	81.37	552.50	
...	...	...	...	...	...	...	...	...	
3525	2023-08-25	84.48	554.00	2123.50	450.36	899.65	84.48	554.00	
3526	2023-08-28	84.42	549.00	2116.25	469.03	933.20	84.42	549.00	
3527	2023-08-29	85.49	545.33	2116.25	460.42	901.79	85.49	545.33	
3528	2023-08-30	85.86	543.67	2140.25	464.80	945.40	85.86	543.67	
3529	2023-08-31	86.86	545.00	2165.25	451.73	955.35	86.86	545.00	

3530 rows × 9 columns

In [42]:
```
df_merge.columns
```

Out[42]:
```
Index(['Date', 'Price_X', 'Price_Y', 'Price_Z', 'Price_Equipo1',
       'Price_Equipo2', 'Price_X2', 'Price_Y2', 'Price_Z2'],
      dtype='object')
```

Conteo de Nulos

In [43]:
```
df_merge.isnull().sum()
```

Out[43]:
```
Date             0
Price_X          0
Price_Y          0
Price_Z          0
Price_Equipo1    0
Price_Equipo2    0
Price_X2         0
Price_Y2         0
Price_Z2         0
dtype: int64
```

Hallar las diferencias entre las bases

In [44]:
```
df_merge['Var_X'] = df_merge['Price_X2'] - df_merge['Price_X']
df_merge['Var_Y'] = df_merge['Price_Y2'] - df_merge['Price_Y']
df_merge['Var_Z'] = df_merge['Price_Z2'] - df_merge['Price_Z']
```

In [45]:
```
df_merge.describe()
```

Out[45]:

	Price_X	Price_Y	Price_Z	Price_Equipo1	Price_Equipo2	Price_X2
count	3530.000000	3530.000000	3530.000000	3530.000000	3530.000000	3530.000000
mean	78.091014	555.527108	2037.432073	460.036901	889.980408	78.091014
std	25.186945	138.489003	373.140448	113.681273	170.042689	25.186945
min	19.330000	257.500000	1421.500000	208.340000	566.000000	19.330000
25%	57.052500	482.500000	1767.250000	398.227500	777.702500	57.052500
50%	75.400000	541.625000	1974.750000	451.250000	869.785000	75.400000
75%	104.582500	620.000000	2235.937500	515.607500	979.122500	104.582500
max	127.980000	1062.370000	3984.000000	855.320000	1703.960000	127.980000

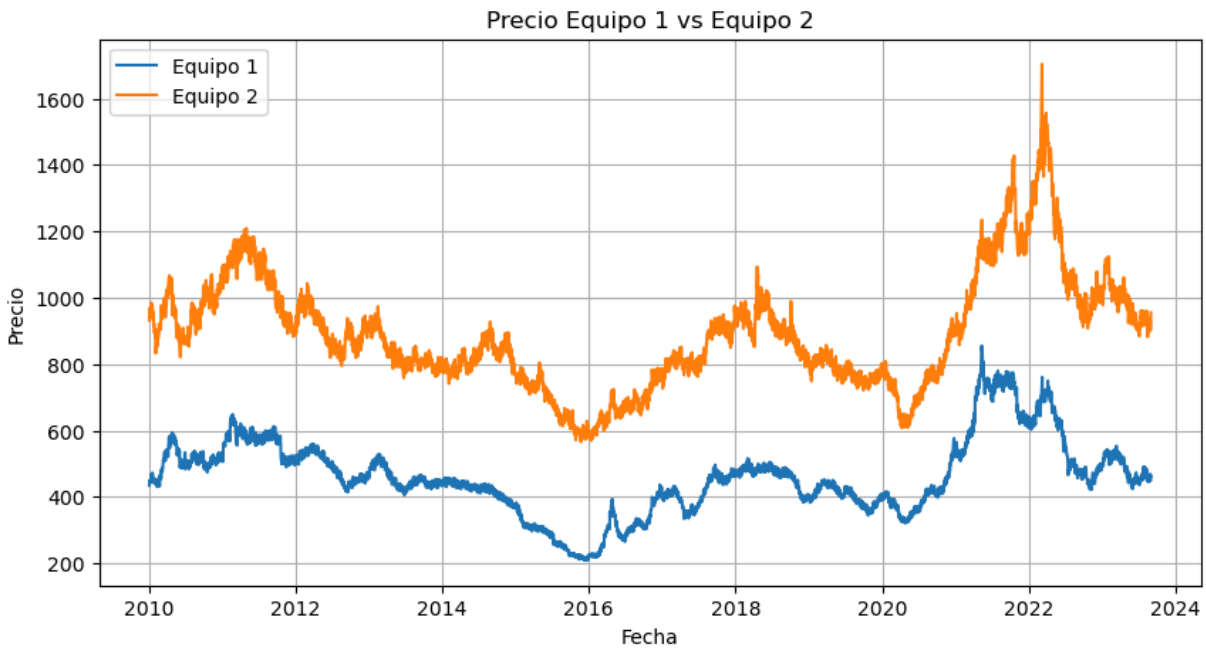
Con este describe, corroboramos que tanto las base de "equipos" como las bases de las materias primas tienen la misma informacion, la diferencia esta en las temporalidades de todas las bases

In [46]:
```
df = equipos.copy()
```

*Las materias primas estan correlacionadas con los precios de los equipos, en especial el Price Y y Price X con el precio del equipo 1 y 2 *El analisis de Correlaciones nos muestra una alta correlacion entre Price Z y Price Y, lo cual nos empieza a indicar multicolinealidad entre ambas *Descartaremos multicolinealidad mediante regresiones

Analisis de las Variables Ovjetivo a traves del tiempo

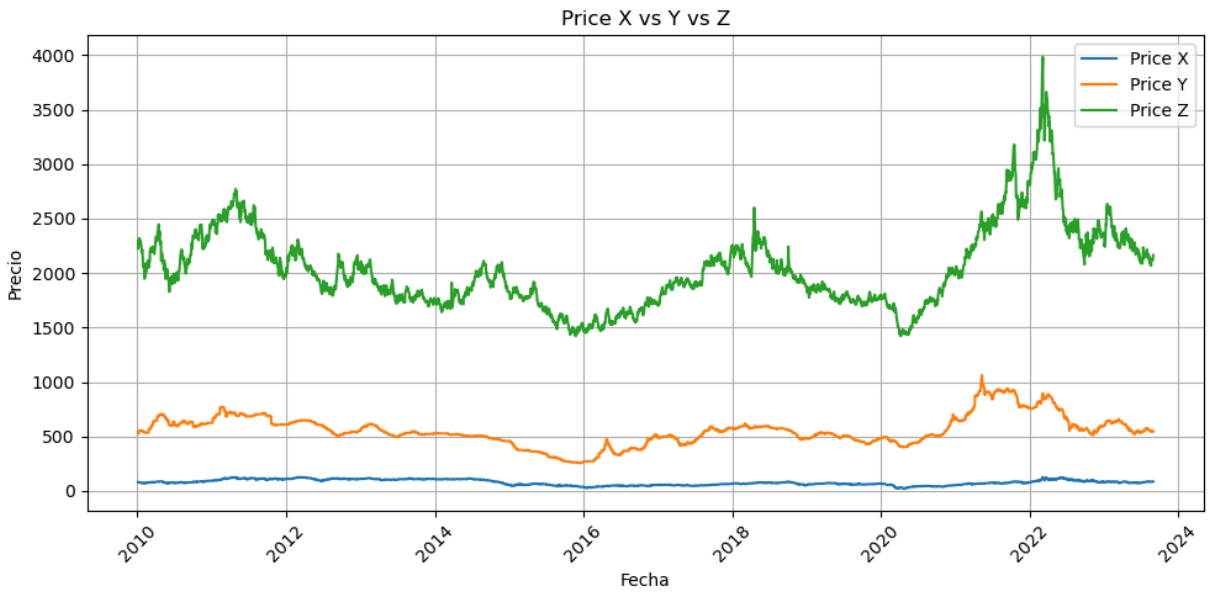
In [47]:
```
import matplotlib.pyplot as plt
import pandas as pd

# Asegúrate de que la columna Date sea tipo datetime
df['Date'] = pd.to_datetime(df['Date'])

# Ordenar por fecha (por si acaso)
df = df.sort_values('Date')

# Crear la gráfica
plt.figure(figsize=(10,5))
plt.plot(df['Date'], df['Price_Equipo1'], label='Equipo 1')
plt.plot(df['Date'], df['Price_Equipo2'], label='Equipo 2')

# Personalización
plt.xlabel('Fecha')
plt.ylabel('Precio')
plt.title('Precio Equipo 1 vs Equipo 2')
plt.legend()
plt.grid()

# Mostrar
plt.show()
```



Se evidencia una posible estacionalidad entre los precios del equipo 1 y 2.

In [48]:
```
import matplotlib.pyplot as plt
import pandas as pd

# Asegurar formato de fecha
df['Date'] = pd.to_datetime(df['Date'])

# Ordenar por fecha
df = df.sort_values('Date')
```

```
# Crear la gráfica
plt.figure(figsize=(10,5))
plt.plot(df['Date'], df['Price_X'], label='Price X')
plt.plot(df['Date'], df['Price_Y'], label='Price Y')
plt.plot(df['Date'], df['Price_Z'], label='Price Z')

# Personalización
plt.xlabel('Fecha')
plt.ylabel('Precio')
plt.title('Price X vs Y vs Z')
plt.legend()
plt.grid()

# Mejorar visual
plt.xticks(rotation=45)
plt.tight_layout()

# Mostrar
plt.show()
```



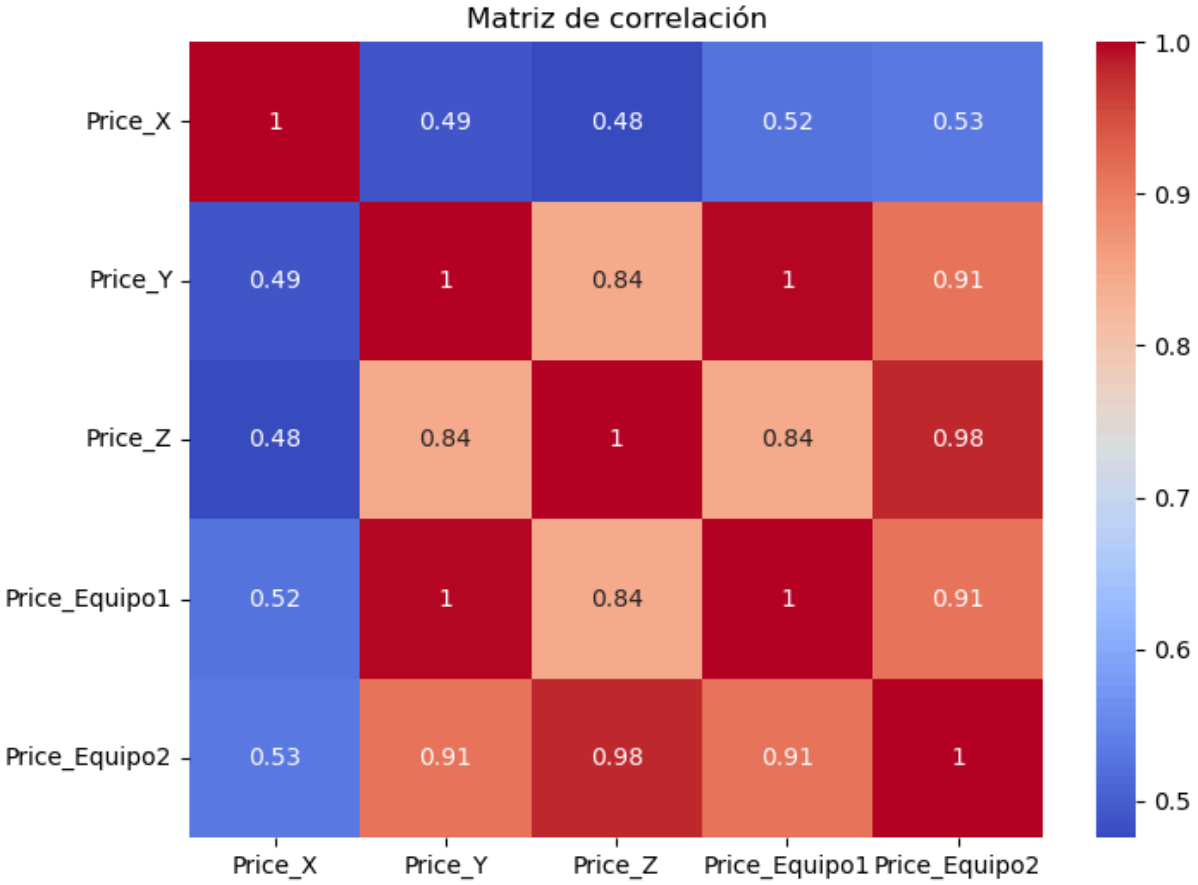
*El comportamiento del Price Z y Y tienden a tener un comportamiento estacional, se realizaran pruebas de Dickey Fuller o Ruido Blanco para establecer si tienen estacionalidad. *El Price Zs el mas elevado y se parece bastante al de los equipos, eso explicaria el por que estan tan correlacionadas. *El Price Y tiene un comportamiento parecido al Price Z, explicando la multiclinealidad.

Analisis de Correlaciones

In [49]:
```
cols = ["Price_X", "Price_Y", "Price_Z", "Price_Equipo1", "Price_Equipo2"]
```

In [50]:
```
import seaborn as sns

corr = df[cols].corr()

plt.figure(figsize=(8,6))
```

```
sns.heatmap(corr, annot=True, cmap="coolwarm")
plt.title("Matriz de correlación")
plt.show()
```



*Las materias primas estan correlacionadas con los precios de los equipos, en especial el Price Y y Price X con el precio del equipo 1 y 2 *El analisis de Correlaciones nos muestra una alta correlacion entre Price Z y Price Y, lo cual nos empieza a indicar multicolinealidad entre ambas *Descartaremos multicolinealidad mediante regresiones

Regresion OLS con todas las materias primas

In [51]:
```
import statsmodels.api as sm

X2 = df[["Price_X", "Price_Y", "Price_Z"]]
X2 = sm.add_constant(X2)

y = df["Price_Equipo2"]

model = sm.OLS(y, X2).fit()
print(model.summary())
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:          Price_Equipo2   R-squared:                       0.992
Model:                            OLS   Adj. R-squared:                  0.992
Method:                 Least Squares   F-statistic:                 1.374e+05
Date:                lun, 20 abr. 2026   Prob (F-statistic):               0.00
Time:                        20:52:36   Log-Likelihood:                -14720.
No. Observations:                3530   AIC:                         2.945e+04
Df Residuals:                    3526   BIC:                         2.947e+04
Df Model:                           3
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
const          1.2278      1.508      0.814      0.416      -1.729       4.185
Price_X        0.3333      0.012     27.482      0.000       0.310       0.357
Price_Y        0.3362      0.004     92.906      0.000       0.329       0.343
Price_Z        0.3318      0.001    249.474      0.000       0.329       0.334
==============================================================================
Omnibus:                      578.827   Durbin-Watson:                   2.040
Prob(Omnibus):                  0.000   Jarque-Bera (JB):              131.046
Skew:                           0.052   Prob(JB):                     3.50e-29
Kurtosis:                       2.062   Cond. No.                     1.23e+04
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly spe
cified.
[2] The condition number is large, 1.23e+04. This might indicate that there are
strong multicollinearity or other numerical problems.
```

Regresion sin materia sin Price Y

In [52]:
```python
X2 = df[["Price_X", "Price_Z"]]
X2 = sm.add_constant(X2)

y = df["Price_Equipo2"]

model = sm.OLS(y, X2).fit()
print(model.summary())
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:          Price_Equipo2   R-squared:                       0.971
Model:                            OLS   Adj. R-squared:                  0.971
Method:                 Least Squares   F-statistic:                 5.854e+04
Date:                lun, 20 abr. 2026   Prob (F-statistic):               0.00
Time:                        20:52:36   Log-Likelihood:                -16905.
No. Observations:                3530   AIC:                         3.382e+04
Df Residuals:                    3527   BIC:                         3.383e+04
Df Model:                           2
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
const        -29.3512      2.732    -10.742      0.000     -34.708     -23.994
Price_X        0.5484      0.022     24.814      0.000       0.505       0.592
Price_Z        0.4302      0.001    288.366      0.000       0.427       0.433
==============================================================================
Omnibus:                      295.826   Durbin-Watson:                   0.606
Prob(Omnibus):                  0.000   Jarque-Bera (JB):              587.905
Skew:                           0.560   Prob(JB):                    2.18e-128
Kurtosis:                       4.657   Cond. No.                     1.16e+04
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly spe
cified.
[2] The condition number is large, 1.16e+04. This might indicate that there are
strong multicollinearity or other numerical problems.
```

Como se puede observar en los coefientes, Price Y está absorbiendo parte del efecto de Z, se elimina Y para evitar multicolinealidad.

In [53]:
```
features = ["Price_X", "Price_Z"]  # Y descartada
target1 = "Price_Equipo1"
target2 = "Price_Equipo2"
```

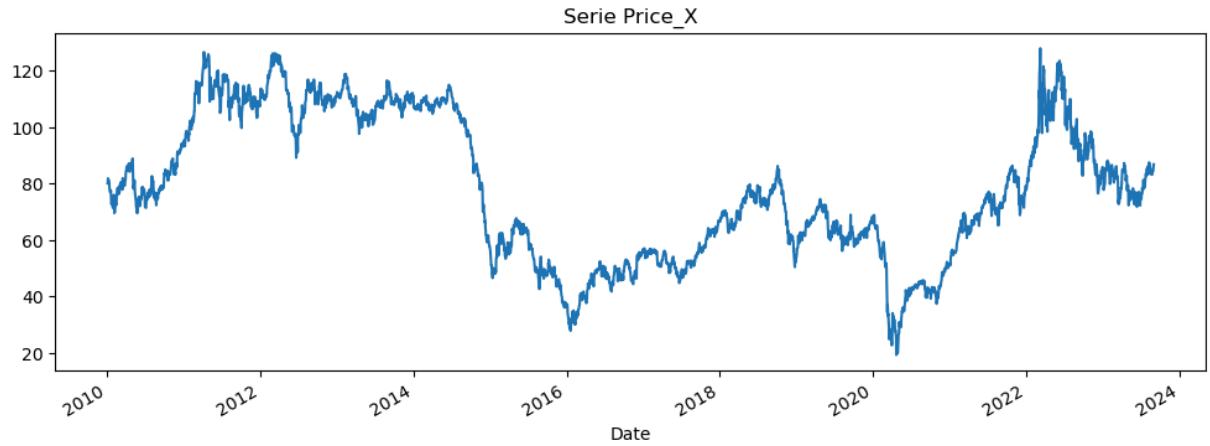
In [54]:
```
# df["X_forecast"] = df["Price_X"].rolling(window=3).mean()
# df["Z_forecast"] = df["Price_Z"].rolling(window=3).mean()
```

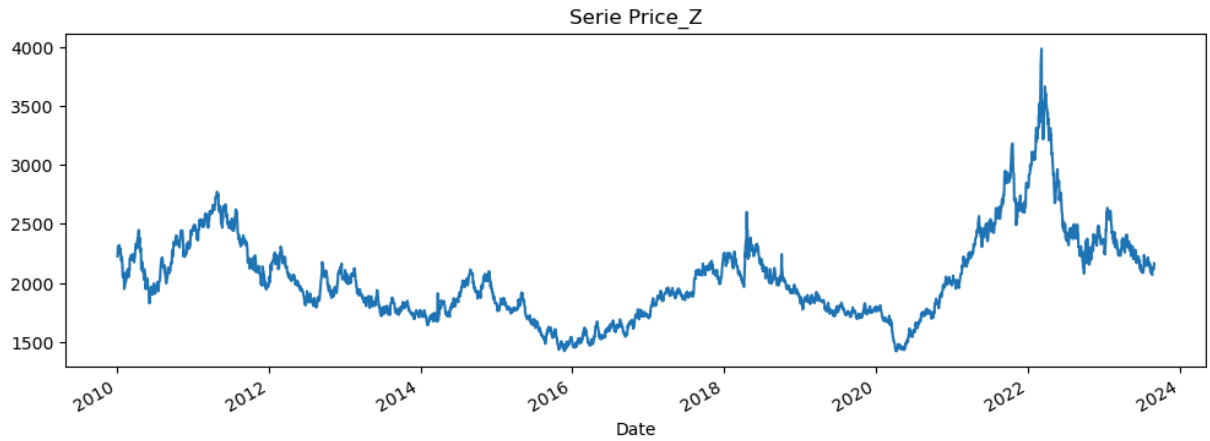
In [55]:
```
# df["Equipo1_model"] = 0.95 * df["Price_Z"]
# df["Equipo2_model"] = 0.7 * df["Price_X"] + 0.3 * df["Price_Z"]
```

Estacionalidad

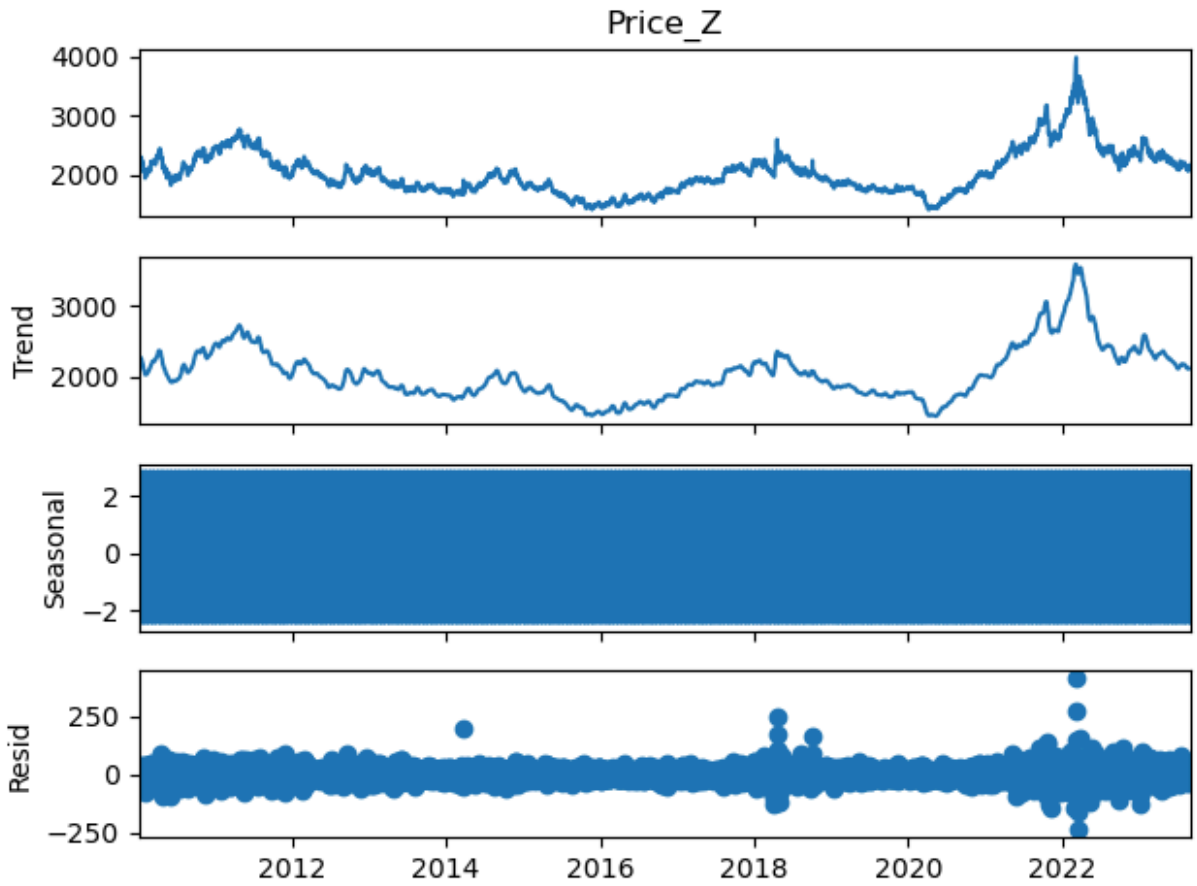
In [56]:
```
import pandas as pd

# Asegurar formato correcto
df["Date"] = pd.to_datetime(df["Date"])
df = df.sort_values("Date")

# Index temporal (clave para ARIMA/SARIMA)
df = df.set_index("Date")
```

In [57]:
```
import matplotlib.pyplot as plt

df["Price_X"].plot(figsize=(12,4), title="Serie Price_X")
plt.show()

df["Price_Z"].plot(figsize=(12,4), title="Serie Price_Z")
plt.show()
```





In [58]:
```
from statsmodels.tsa.seasonal import seasonal_decompose

# Ejemplo con Z
decomp = seasonal_decompose(df["Price_Z"], model='additive', period=12)

decomp.plot()
plt.show()
```



In [59]:
```
from statsmodels.tsa.stattools import adfuller

def adf_test(series, name=""):
    result = adfuller(series.dropna())
    print(f"ADF Test {name}")
    print("p-value:", result[1])

adf_test(df["Price_X"], "X")
adf_test(df["Price_Z"], "Z")
```

```
ADF Test X
p-value: 0.42843847586600803
ADF Test Z
p-value: 0.20519131055447198
```

In [60]:
```
df["X_diff"] = df["Price_X"].diff()
df["Z_diff"] = df["Price_Z"].diff()
```

In [61]:
```
adf_test(df["X_diff"].dropna(), "X diff")
adf_test(df["Z_diff"].dropna(), "Z diff")
```

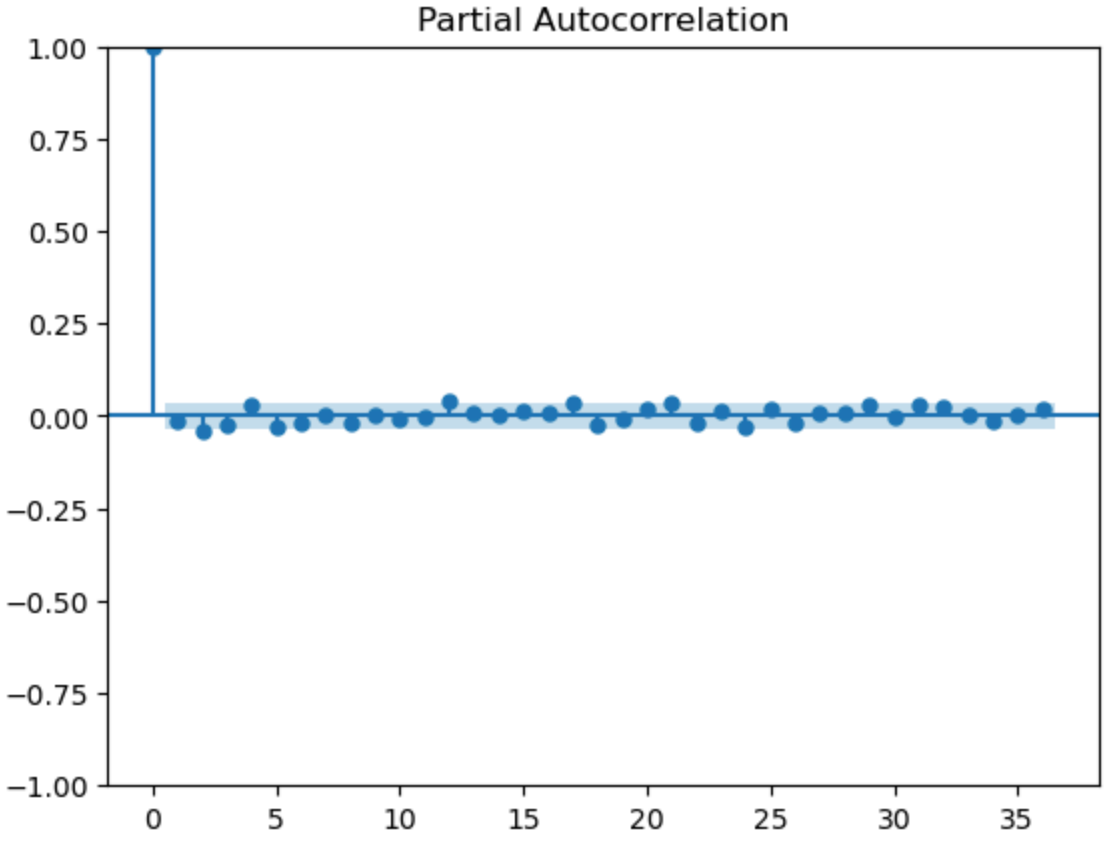
```
ADF Test X diff
p-value: 0.0
ADF Test Z diff
p-value: 2.723942677234721e-27
```

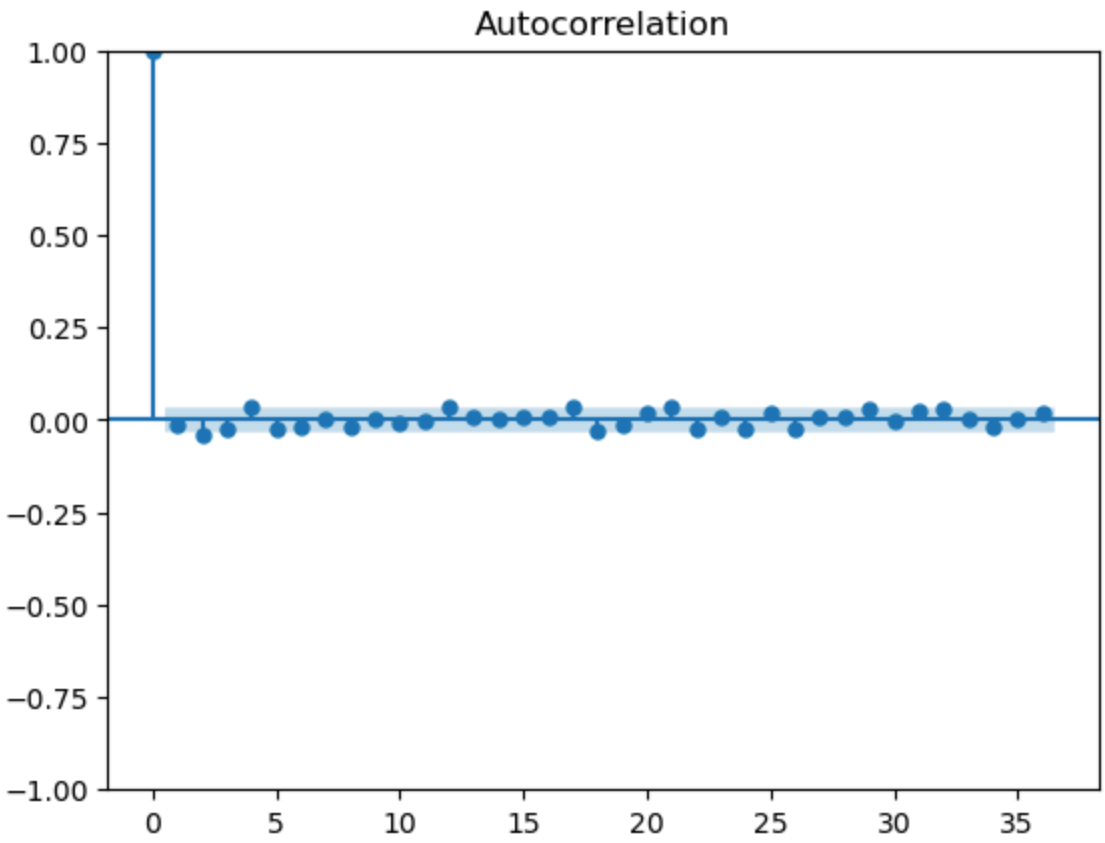
Se realizó la prueba de Dickey-Fuller aumentada (ADF) para evaluar la estacionariedad de las series de precios de las materias primas. Los resultados mostraron p-values superiores a

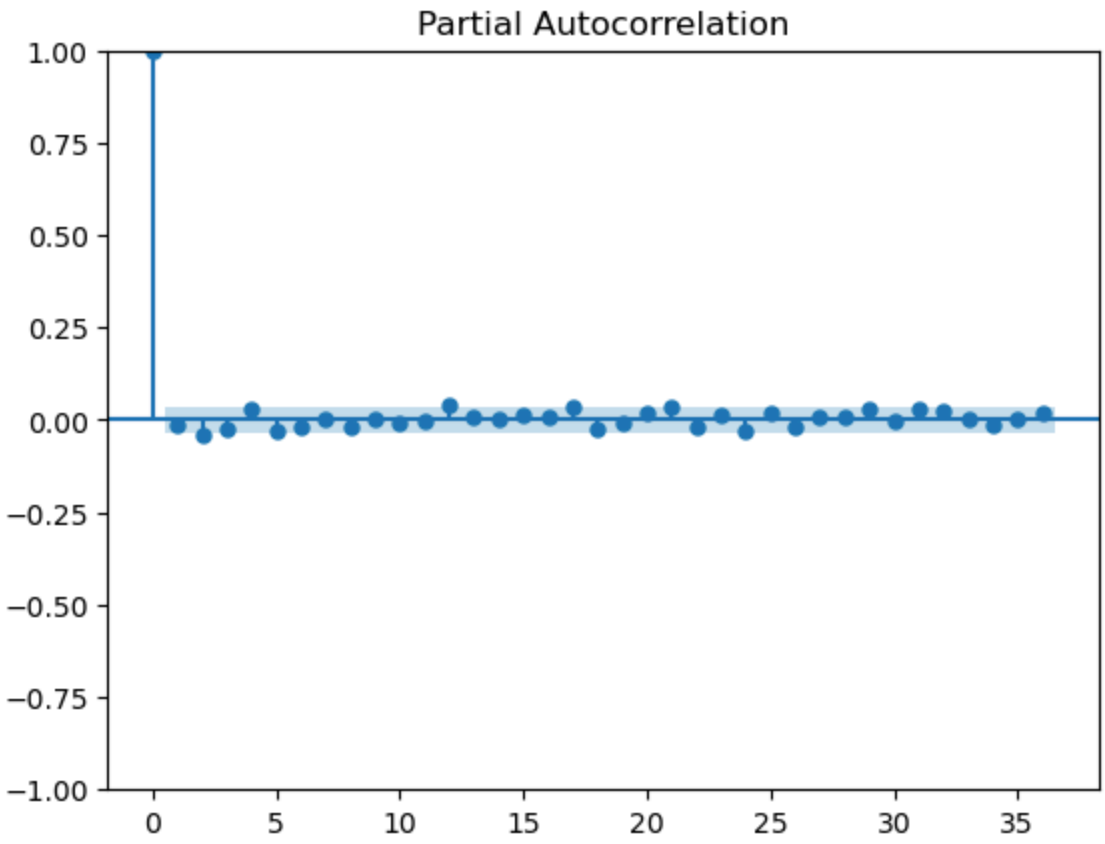
0.05, indicando no estacionariedad. En consecuencia, se aplicó una diferenciación de primer orden (d=1) para estabilizar las series antes de ajustar modelos ARIMA/SARIMA.

In [62]:
```
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf

plot_acf(df["X_diff"].dropna())
plot_pacf(df["X_diff"].dropna())
```

Out[62]:







Las gráficas ACF y PACF muestran la diferenciación, la serie no presenta autocorrelaciones significativas, lo que indica estacionariedad. Esto sugiere que la dinámica de la serie puede ser capturada con modelos de bajo orden. Se optó por un SARIMA(1,1,1) con componente

estacional para capturar posibles patrones débiles y estacionales, logrando un desempeño predictivo alto.

Modelo SARIMAX

In [63]:
```
from statsmodels.tsa.statespace.sarimax import SARIMAX

# Ajusta órdenes según análisis (ejemplo base)
model_X = SARIMAX(df["Price_X"], order=(1,1,1), seasonal_order=(1,1,1,12))
results_X = model_X.fit()

print(results_X.summary())
```

```
c:\Users\Sebastian.garzon.LAPTOP-G9J39UDS\anaconda3\Lib\site-packages\statsmodels\ts
a\base\tsa_model.py:473: ValueWarning: A date index has been provided, but it has no
associated frequency information and so will be ignored when e.g. forecasting.
  self._init_dates(dates, freq)
c:\Users\Sebastian.garzon.LAPTOP-G9J39UDS\anaconda3\Lib\site-packages\statsmodels\ts
a\base\tsa_model.py:473: ValueWarning: A date index has been provided, but it has no
associated frequency information and so will be ignored when e.g. forecasting.
  self._init_dates(dates, freq)
c:\Users\Sebastian.garzon.LAPTOP-G9J39UDS\anaconda3\Lib\site-packages\statsmodels\ts
a\statespace\sarimax.py:966: UserWarning: Non-stationary starting autoregressive par
ameters found. Using zeros as starting parameters.
  warn('Non-stationary starting autoregressive parameters'
c:\Users\Sebastian.garzon.LAPTOP-G9J39UDS\anaconda3\Lib\site-packages\statsmodels\ts
a\statespace\sarimax.py:978: UserWarning: Non-invertible starting MA parameters foun
d. Using zeros as starting parameters.
  warn('Non-invertible starting MA parameters found.'
```

```
                                     SARIMAX Results
==================================================================================
======
Dep. Variable:                            Price_X   No. Observations:
3530
Model:             SARIMAX(1, 1, 1)x(1, 1, 1, 12)   Log Likelihood               -65
92.427
Date:                           lun, 20 abr. 2026   AIC                          131
94.853
Time:                                   20:52:45   BIC                          132
25.680
Sample:                                        0   HQIC                         132
05.852
                                          - 3530
Covariance Type:                             opg
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
ar.L1          0.7560      0.139      5.454      0.000       0.484       1.028
ma.L1         -0.7805      0.132     -5.899      0.000      -1.040      -0.521
ar.S.L12       0.0364      0.013      2.894      0.004       0.012       0.061
ma.S.L12      -0.9999      0.149     -6.727      0.000      -1.291      -0.709
sigma2         2.4400      0.358      6.821      0.000       1.739       3.141
===================================================================================
Ljung-Box (L1) (Q):                   0.52   Jarque-Bera (JB):             10398.41
Prob(Q):                              0.47   Prob(JB):                         0.00
Heteroskedasticity (H):               1.64   Skew:                            -0.79
Prob(H) (two-sided):                  0.00   Kurtosis:                        11.27
===================================================================================

Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-ste
p).
```

In [64]:

```python
model_Z = SARIMAX(df["Price_Z"], order=(1,1,1), seasonal_order=(1,1,1,12))
results_Z = model_Z.fit()

print(results_Z.summary())
```

```
c:\Users\Sebastian.garzon.LAPTOP-G9J39UDS\anaconda3\Lib\site-packages\statsmodels\ts
a\base\tsa_model.py:473: ValueWarning: A date index has been provided, but it has no
associated frequency information and so will be ignored when e.g. forecasting.
  self._init_dates(dates, freq)
c:\Users\Sebastian.garzon.LAPTOP-G9J39UDS\anaconda3\Lib\site-packages\statsmodels\ts
a\base\tsa_model.py:473: ValueWarning: A date index has been provided, but it has no
associated frequency information and so will be ignored when e.g. forecasting.
  self._init_dates(dates, freq)
```

```
                                     SARIMAX Results
==================================================================================
======
Dep. Variable:                            Price_Z   No. Observations:
3530
Model:             SARIMAX(1, 1, 1)x(1, 1, 1, 12)   Log Likelihood              -171
10.497
Date:                           lun, 20 abr. 2026   AIC                          342
30.993
Time:                                   20:52:50   BIC                          342
61.820
Sample:                                        0   HQIC                         342
41.992
                                          - 3530
Covariance Type:                             opg
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
ar.L1         -0.5141      0.106     -4.827      0.000      -0.723      -0.305
ma.L1          0.4472      0.111      4.021      0.000       0.229       0.665
ar.S.L12       0.0256      0.010      2.568      0.010       0.006       0.045
ma.S.L12      -0.9986      0.015    -65.978      0.000      -1.028      -0.969
sigma2       967.0162     15.033     64.327      0.000     937.552     996.480
===================================================================================
Ljung-Box (L1) (Q):                   0.12   Jarque-Bera (JB):             64537.24
Prob(Q):                              0.73   Prob(JB):                         0.00
Heteroskedasticity (H):               1.79   Skew:                            -1.08
Prob(H) (two-sided):                  0.00   Kurtosis:                        23.87
===================================================================================

Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-ste
p).
```

Proyectamos la serie a 30 periodos en el futuro (1 mes), debido a que los datos son diario, se manejo esta temporalidad para evaluar el nivel de prediccion de las materias primas.

In [65]:
```
steps = 30

forecast_X = results_X.get_forecast(steps=steps)
forecast_Z = results_Z.get_forecast(steps=steps)

X_pred = forecast_X.predicted_mean
Z_pred = forecast_Z.predicted_mean
```

```
c:\Users\Sebastian.garzon.LAPTOP-G9J39UDS\anaconda3\Lib\site-packages\statsmodels\ts
a\base\tsa_model.py:837: ValueWarning: No supported index is available. Prediction r
esults will be given with an integer index beginning at `start`.
  return get_prediction_index(
c:\Users\Sebastian.garzon.LAPTOP-G9J39UDS\anaconda3\Lib\site-packages\statsmodels\ts
a\base\tsa_model.py:837: FutureWarning: No supported index is available. In the next
version, calling this method in a model without a supported index will result in an
exception.
  return get_prediction_index(
c:\Users\Sebastian.garzon.LAPTOP-G9J39UDS\anaconda3\Lib\site-packages\statsmodels\ts
a\base\tsa_model.py:837: ValueWarning: No supported index is available. Prediction r
esults will be given with an integer index beginning at `start`.
  return get_prediction_index(
```

In [66]:
```
conf_X = forecast_X.conf_int()
conf_Z = forecast_Z.conf_int()
```

In [67]:
```
pred_x_conso = pd.concat([conf_X,X_pred], axis = 1).reset_index(drop = True)
```

intervalos de confianza del forestas SARIMAX

In [68]:
```
pred_x_conso
```

Out[68]:

	lower Price_X	upper Price_X	predicted_mean
0	83.435610	89.568716	86.502163
1	82.219386	90.787447	86.503417
2	81.398040	91.783780	86.590910
3	80.519488	92.407129	86.463308
4	79.840891	93.032653	86.436772
5	79.044770	93.403420	86.224095
6	78.642404	94.066419	86.354412
7	78.222221	94.633035	86.427628
8	77.938004	95.272719	86.605362
9	77.520657	95.727562	86.624109
10	77.227952	96.263597	86.745775
11	76.852786	96.679994	86.766390
12	76.166681	96.820131	86.493406
13	75.782298	97.227167	86.504732
14	75.493726	97.699631	86.596679
15	75.026408	97.966510	86.496459
16	74.674287	98.324628	86.499458
17	74.150603	98.489604	86.320104
18	73.954163	98.962233	86.458198
19	73.670149	99.329379	86.499764
20	73.544036	99.837961	86.690998
21	73.218481	100.131883	86.675182
22	73.031215	100.549965	86.790590
23	72.722231	100.833135	86.777683
24	72.161029	100.857713	86.509371
25	71.887000	101.157524	86.522262
26	71.698694	101.531859	86.615277
27	71.324104	101.709373	86.516738
28	71.057615	101.985043	86.521329
29	70.613488	102.073668	86.343578

In [69]:
```
X
```

Out[69]:

	Date	Price
0	2024-04-04	89.18
1	2024-04-03	89.35
2	2024-04-02	88.92
3	2024-04-01	87.42
4	2024-03-28	87.48
...	...	...
9139	1988-07-01	14.60
9140	1988-06-30	14.85
9141	1988-06-29	15.47
9142	1988-06-28	15.27
9143	1988-06-27	15.10

9144 rows × 2 columns

In [70]:
```
real = X[X["Date"] >= equipos["Date"].max()].head(30)
```

In [71]:
```
pd.concat([real,X_pred.reset_index(drop=True)], axis = 1)
```

Out[71]:

	Date	Price	predicted_mean
0	2024-04-04	89.18	86.502163
1	2024-04-03	89.35	86.503417
2	2024-04-02	88.92	86.590910
3	2024-04-01	87.42	86.463308
4	2024-03-28	87.48	86.436772
5	2024-03-27	86.09	86.224095
6	2024-03-26	86.25	86.354412
7	2024-03-25	86.75	86.427628
8	2024-03-22	85.43	86.605362
9	2024-03-21	85.78	86.624109
10	2024-03-20	85.95	86.745775
11	2024-03-19	87.38	86.766390
12	2024-03-18	86.89	86.493406
13	2024-03-15	85.34	86.504732
14	2024-03-14	85.42	86.596679
15	2024-03-13	84.03	86.496459
16	2024-03-12	81.92	86.499458
17	2024-03-11	82.21	86.320104
18	2024-03-08	82.08	86.458198
19	2024-03-07	82.96	86.499764
20	2024-03-06	82.96	86.690998
21	2024-03-05	82.04	86.675182
22	2024-03-04	82.80	86.790590
23	2024-03-01	83.55	86.777683
24	2024-02-29	83.62	86.509371
25	2024-02-28	83.68	86.522262
26	2024-02-27	83.65	86.615277
27	2024-02-26	82.53	86.516738
28	2024-02-23	81.62	86.521329
29	2024-02-22	83.67	86.343578

In [72]:
```
from sklearn.metrics import mean_squared_error, mean_absolute_percentage_error


y_true = real["Price"]
y_pred = X_pred

# MSE
mse = mean_squared_error(y_true, y_pred)

# MAPE
mape = mean_absolute_percentage_error(y_true, y_pred)

print("MSE:", mse)
print("MAPE:", mape)
```

```
MSE: 7.950546914280651
MAPE: 0.028437875245016894
```

- Price X tenia periodos despues de la fecha de la base de equipo, esta ventaja la use para verificar que el modelo SARIMAX, estuviera prediciendo bien.
- El modelo presenta un muy buen desempeño. El MSE (7.95) indica que los errores son bajos y no hay grandes desviaciones, mientras que el MAPE (2.84%) muestra que, en promedio, el error es menor al 3%, lo cual se considera excelente.
- En consecuencia, el modelo tiene un alto poder predictivo y es confiable para proyectar el precio de X.

Promedio Movil

Creamos variables de promedios móviles con el objetivo de mejorar la capacidad predictiva de los modelos para los equipos 1 y 2. Para ello, utilizamos diferentes ventanas de tiempo, buscando capturar patrones y tendencias que aportaran mayor información y robustez a las predicciones.

In [73]:
```
df["X_prom_3"] = df["Price_X"].rolling(window=3).mean()
df["Z_prom_3"] = df["Price_Z"].rolling(window=3).mean()
df["X_prom_6"] = df["Price_X"].rolling(window=6).mean()
df["Z_prom_6"] = df["Price_Z"].rolling(window=6).mean()
df["X_prom_9"] = df["Price_X"].rolling(window=9).mean()
df["Z_prom_9"] = df["Price_Z"].rolling(window=9).mean()
df["X_prom_12"] = df["Price_X"].rolling(window=12).mean()
df["Z_prom_12"] = df["Price_Z"].rolling(window=12).mean()
df["X_prom_15"] = df["Price_X"].rolling(window=15).mean()
df["Z_prom_15"] = df["Price_Z"].rolling(window=15).mean()
df["X_prom_30"] = df["Price_X"].rolling(window=30).mean()
df["Z_prom_30"] = df["Price_Z"].rolling(window=30).mean()
```

In [74]:
```
df
```

Out[74]:

Date	Price_X	Price_Y	Price_Z	Price_Equipo1	Price_Equipo2	X_diff	Z_diff	X_prom_3	
2010-01-04	80.12	527.50	2225.25	434.73	931.73	NaN	NaN	NaN	
2010-01-05	80.59	527.50	2246.50	449.97	968.56	0.47	21.25	NaN	
2010-01-06	81.89	527.50	2302.50	444.48	960.51	1.30	56.00	80.866667	2
2010-01-07	81.51	527.50	2306.50	440.90	960.14	-0.38	4.00	81.330000	2
2010-01-08	81.37	552.50	2261.25	448.82	949.55	-0.14	-45.25	81.590000	2
...	...	...	...	...	...	...	...	...	
2023-08-25	84.48	554.00	2123.50	450.36	899.65	1.12	4.25	83.683333	2
2023-08-28	84.42	549.00	2116.25	469.03	933.20	-0.06	-7.25	84.086667	2
2023-08-29	85.49	545.33	2116.25	460.42	901.79	1.07	0.00	84.796667	2
2023-08-30	85.86	543.67	2140.25	464.80	945.40	0.37	24.00	85.256667	2
2023-08-31	86.86	545.00	2165.25	451.73	955.35	1.00	25.00	86.070000	2

3530 rows × 19 columns

Precios Lag

Construimos variables rezagadas (lags) con el fin de evaluar si la información histórica de las variables podía aportar mayor capacidad predictiva a los modelos de los equipos 1 y 2. Para ello, incorporamos distintos rezagos en el tiempo, buscando capturar efectos diferidos y dependencias temporales que pudieran mejorar la precisión de las predicciones.

In [75]:

```
# Lags reales (valores pasados)
df["X_lag_1"] = df["Price_X"].shift(1)
df["X_lag_3"] = df["Price_X"].shift(3)
df["X_lag_6"] = df["Price_X"].shift(6)
df["X_lag_12"] = df["Price_X"].shift(12)
df["X_lag_15"] = df["Price_X"].shift(15)
df["X_lag_30"] = df["Price_X"].shift(30)
df["Z_lag_1"] = df["Price_Z"].shift(1)
df["Z_lag_3"] = df["Price_Z"].shift(3)
```

```
df["Z_lag_6"] = df["Price_Z"].shift(6)
df["Z_lag_12"] = df["Price_Z"].shift(12)
df["Z_lag_15"] = df["Price_Z"].shift(15)
df["Z_lag_30"] = df["Price_Z"].shift(30)
```

In [76]:
```
df
```

Out[76]:

Date	Price_X	Price_Y	Price_Z	Price_Equipo1	Price_Equipo2	X_diff	Z_diff	X_prom_3	
2010-01-04	80.12	527.50	2225.25	434.73	931.73	NaN	NaN	NaN	
2010-01-05	80.59	527.50	2246.50	449.97	968.56	0.47	21.25	NaN	
2010-01-06	81.89	527.50	2302.50	444.48	960.51	1.30	56.00	80.866667	2
2010-01-07	81.51	527.50	2306.50	440.90	960.14	-0.38	4.00	81.330000	2
2010-01-08	81.37	552.50	2261.25	448.82	949.55	-0.14	-45.25	81.590000	2
...	...	...	...	...	...	...	...	...	
2023-08-25	84.48	554.00	2123.50	450.36	899.65	1.12	4.25	83.683333	2
2023-08-28	84.42	549.00	2116.25	469.03	933.20	-0.06	-7.25	84.086667	2
2023-08-29	85.49	545.33	2116.25	460.42	901.79	1.07	0.00	84.796667	2
2023-08-30	85.86	543.67	2140.25	464.80	945.40	0.37	24.00	85.256667	2
2023-08-31	86.86	545.00	2165.25	451.73	955.35	1.00	25.00	86.070000	2

3530 rows × 31 columns

In [77]:
```
df.dropna()
```

Out[77]:

Date	Price_X	Price_Y	Price_Z	Price_Equipo1	Price_Equipo2	X_diff	Z_diff	X_prom_3	
2010-02-15	72.51	535.00	2032.50	445.36	869.78	-0.39	28.25	72.820000	2
2010-02-16	75.68	535.00	2050.50	434.38	904.22	3.17	18.00	73.696667	2
2010-02-17	76.27	535.00	2094.50	431.69	877.95	0.59	44.00	74.820000	2
2010-02-18	77.78	535.00	2076.50	455.50	892.05	1.51	-18.00	76.576667	2
2010-02-19	78.19	535.00	2071.50	456.03	892.66	0.41	-5.00	77.413333	2
...	...	...	...	...	...	...	...	...	
2023-08-25	84.48	554.00	2123.50	450.36	899.65	1.12	4.25	83.683333	2
2023-08-28	84.42	549.00	2116.25	469.03	933.20	-0.06	-7.25	84.086667	2
2023-08-29	85.49	545.33	2116.25	460.42	901.79	1.07	0.00	84.796667	2
2023-08-30	85.86	543.67	2140.25	464.80	945.40	0.37	24.00	85.256667	2
2023-08-31	86.86	545.00	2165.25	451.73	955.35	1.00	25.00	86.070000	2

3500 rows × 31 columns

In [78]:
```
## Hallamos solo las Features con las que modelaremos
features = df.reset_index().drop(["Z_diff","X_diff","Date","Price_Y","Price_Equipo2
```

In [81]:
```
features
```

Out[81]:
```
Index(['Price_X', 'Price_Z', 'X_prom_3', 'Z_prom_3', 'X_prom_6', 'Z_prom_6',
       'X_prom_9', 'Z_prom_9', 'X_prom_12', 'Z_prom_12', 'X_prom_15',
       'Z_prom_15', 'X_prom_30', 'Z_prom_30', 'X_lag_1', 'X_lag_3', 'X_lag_6',
       'X_lag_12', 'X_lag_15', 'X_lag_30', 'Z_lag_1', 'Z_lag_3', 'Z_lag_6',
       'Z_lag_12', 'Z_lag_15', 'Z_lag_30'],
      dtype='object')
```

Modelamiento

In [82]:
```
from xgboost import XGBRegressor
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error
```

```python
import numpy as np

# Dataset sin NaN
df_ml = df

X = df_ml[features]

y1 = df_ml[target1]
y2 = df_ml[target2]

X_train, X_test, y1_train, y1_test = train_test_split(X, y1, test_size=0.2, shuffle
_, _, y2_train, y2_test = train_test_split(X, y2, test_size=0.2, shuffle=False)

# Modelo para Equipo 1
model1 = XGBRegressor()
model1.fit(X_train, y1_train)

# Modelo para Equipo 2
model2 = XGBRegressor()
model2.fit(X_train, y2_train)

# Predicciones
y1_pred = model1.predict(X_test)
y2_pred = model2.predict(X_test)

# Métricas
rmse1 = np.sqrt(mean_squared_error(y1_test, y1_pred))
rmse2 = np.sqrt(mean_squared_error(y2_test, y2_pred))

mape1 = mean_absolute_percentage_error(y1_test, y1_pred)
mape2 = mean_absolute_percentage_error(y2_test, y2_pred)

print("📊 RESULTADOS MODELO FINAL")
print("---------------------------------")
print(f"Equipo 1 -> RMSE: {rmse1:.4f} | MAPE: {mape1:.4%}")
print(f"Equipo 2 -> RMSE: {rmse2:.4f} | MAPE: {mape2:.4%}")
```

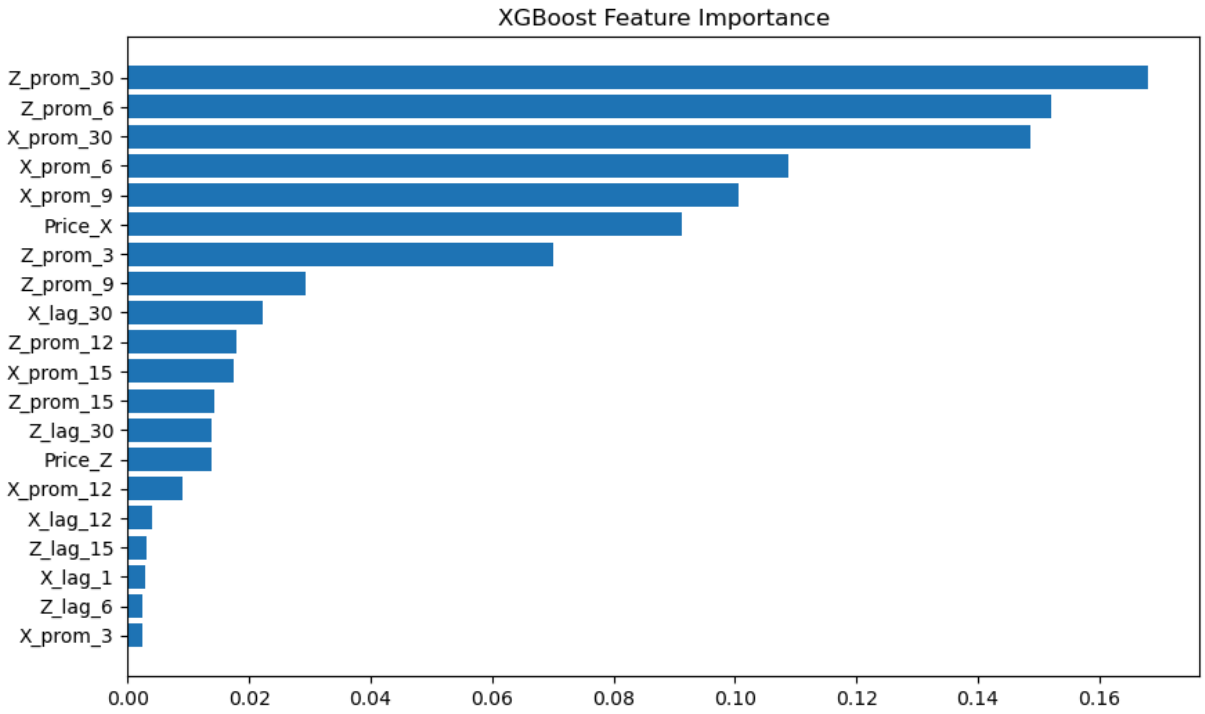
```
📊 RESULTADOS MODELO FINAL
---------------------------------
Equipo 1 -> RMSE: 141.1108 | MAPE: 17.8548%
Equipo 2 -> RMSE: 117.3028 | MAPE: 6.3640%
```

- El modelo final seleccionado utiliza el conjunto completo de variables, logrando un MAPE de 6.36% para el Equipo 2 y 17.85% para el Equipo 1, evidenciando un alto nivel de precisión y capacidad predictiva.
- Intente iterar condo diferentes tipo de combinacion de variables, pero los KPIS empeoraban bastante, por eso decidí dejar todas las variables
- Se Testeo solo con 20% de la base debido a la cantidad tan limitada de datos historicos para modelar

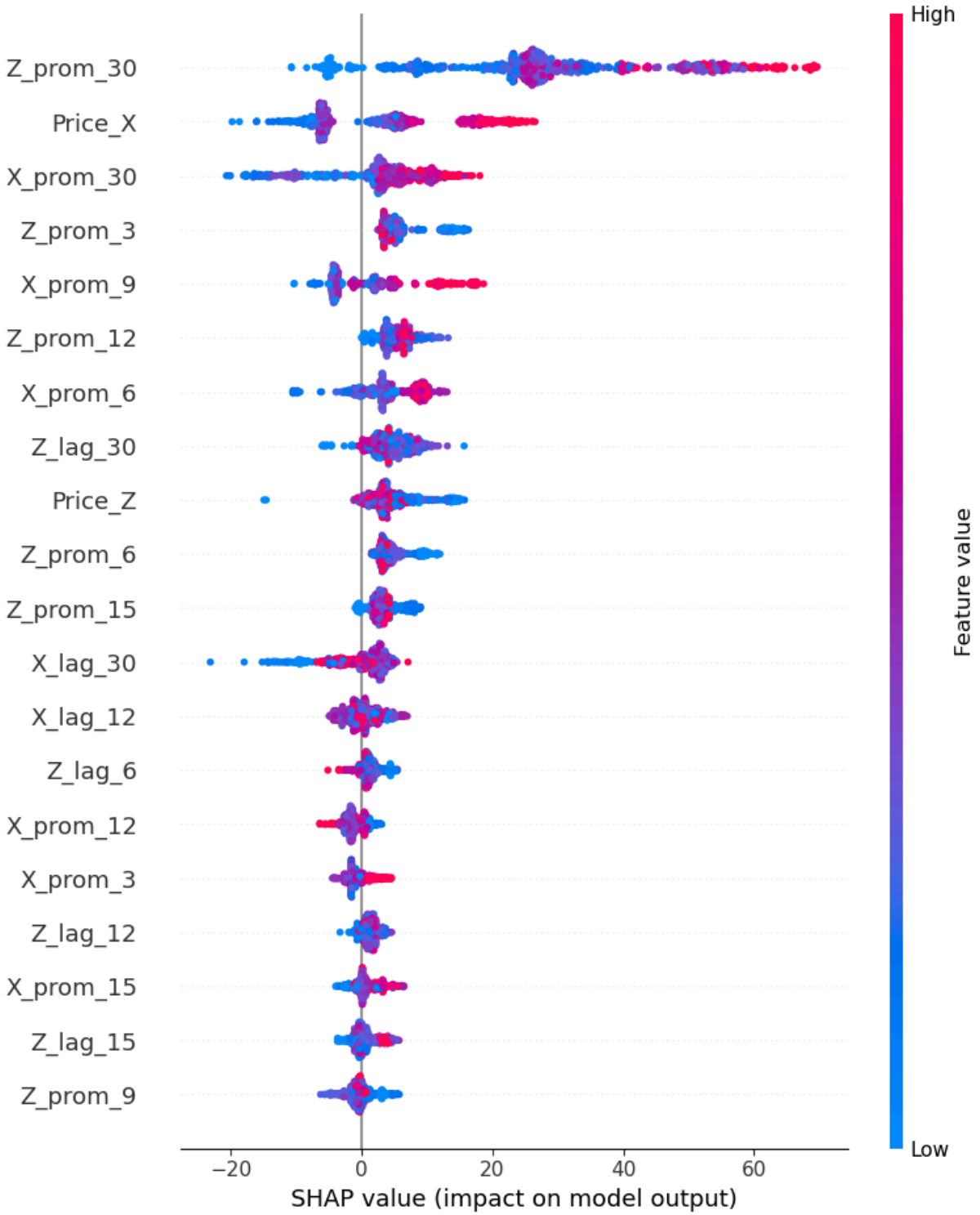
Eleccion de varibles modelo 1 (Precio Equipo 1)

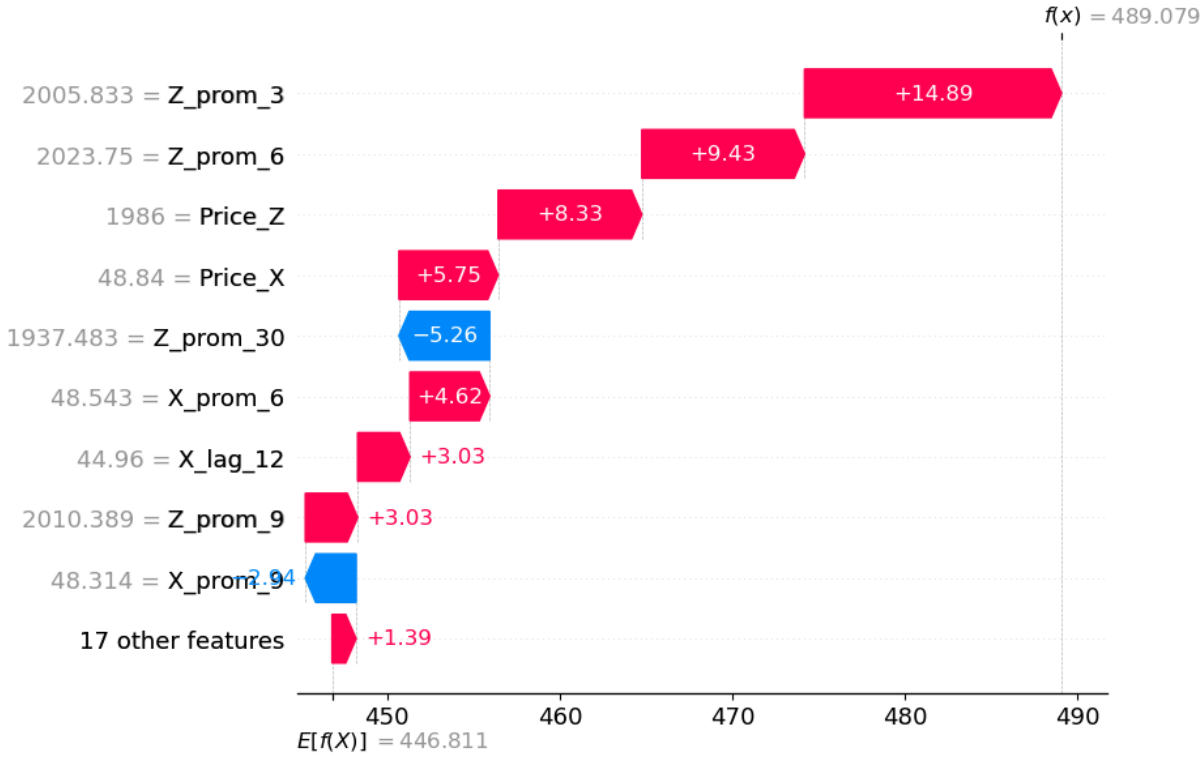
In [83]:
```
importances = model1.feature_importances_
importance_df = pd.DataFrame({
    'Feature': features,
    'Importance': importances
}).sort_values(by='Importance', ascending=False).head(20)

print("\n🎯 Feature Importance (XGBoost):\n", importance_df)

# Graficar
plt.figure(figsize=(10, 6))
plt.barh(importance_df['Feature'], importance_df['Importance'])
plt.gca().invert_yaxis()
plt.title("XGBoost Feature Importance")
plt.show()
```

```
🎯 Feature Importance (XGBoost):
       Feature  Importance
13  Z_prom_30    0.168090
5    Z_prom_6    0.151957
12  X_prom_30    0.148585
4    X_prom_6    0.108846
6    X_prom_9    0.100663
0     Price_X    0.091355
3    Z_prom_3    0.070073
7    Z_prom_9    0.029284
19   X_lag_30    0.022209
9   Z_prom_12    0.017928
10  X_prom_15    0.017503
11  Z_prom_15    0.014221
25   Z_lag_30    0.013830
1     Price_Z    0.013773
8   X_prom_12    0.009052
17   X_lag_12    0.004097
24   Z_lag_15    0.003175
14    X_lag_1    0.002917
22    Z_lag_6    0.002473
2    X_prom_3    0.002356
```



In [86]:
```
import shap

explainer = shap.Explainer(model1, X_train)
shap_values = explainer(X_test)

# Resumen global
shap.summary_plot(shap_values, X_test)

# También puedes ver un resumen por instancia individual:
shap.plots.waterfall(shap_values[0])
```


Z_prom_30
Price_X
X_prom_30
Z_prom_3
X_prom_9
Z_prom_12
X_prom_6
Z_lag_30
Price_Z
Z_prom_6
Z_prom_15
X_lag_30
X_lag_12
Z_lag_6
X_prom_12
X_prom_3
Z_lag_12
X_prom_15
Z_lag_15
Z_prom_9
−20
0
20
40
60
SHAP value (impact on model output)
High
Feature value
Low



In [87]:

```
# Convertir a array
shap_vals = shap_values.values

# Importancia global (media absoluta)
shap_importance = np.abs(shap_vals).mean(axis=0)

# Crear DataFrame
feature_importance = pd.DataFrame({
    "Feature": X_test.columns,
    "Importance": shap_importance
})

# Ordenar
feature_importance = feature_importance.sort_values(by="Importance", ascending=Fals

# Top 10
top12_model1 = feature_importance.head(12)

print(top12_model1)
```

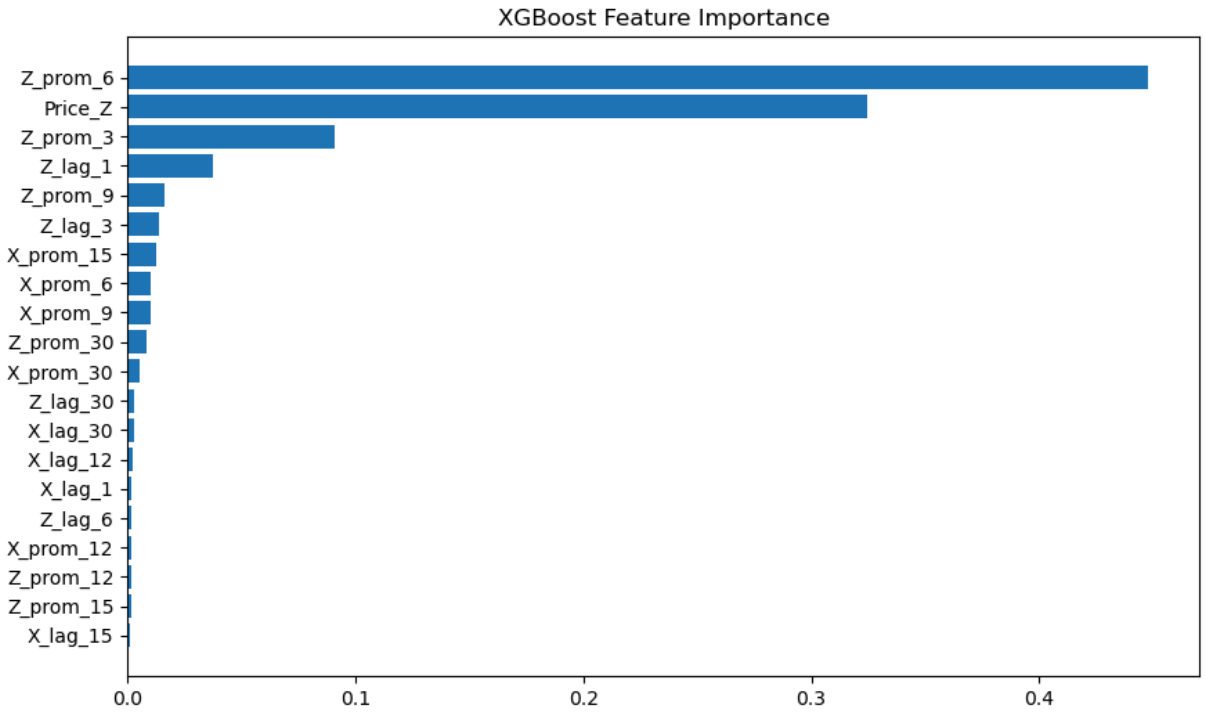
```
      Feature  Importance
13  Z_prom_30   29.410772
0     Price_X    9.205234
12  X_prom_30    6.729336
3    Z_prom_3    5.864222
6    X_prom_9    5.201845
9   Z_prom_12    5.133158
4    X_prom_6    5.111293
25   Z_lag_30    4.702391
1     Price_Z    4.673797
5    Z_prom_6    4.515048
11  Z_prom_15    3.559486
19   X_lag_30    3.123561
```

Eleccion de varibles modelo 2 (Precio Equipo 2)

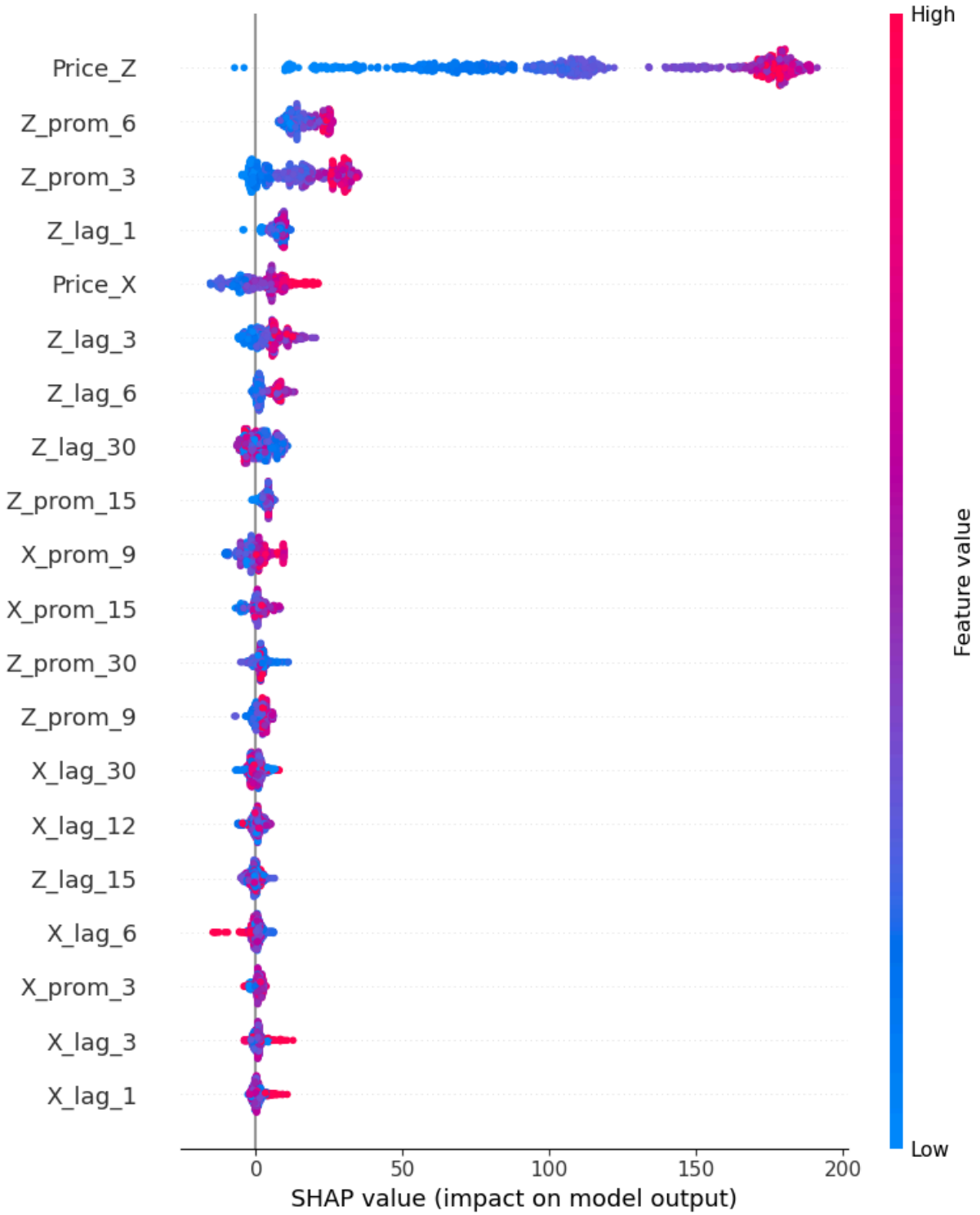
In [88]:

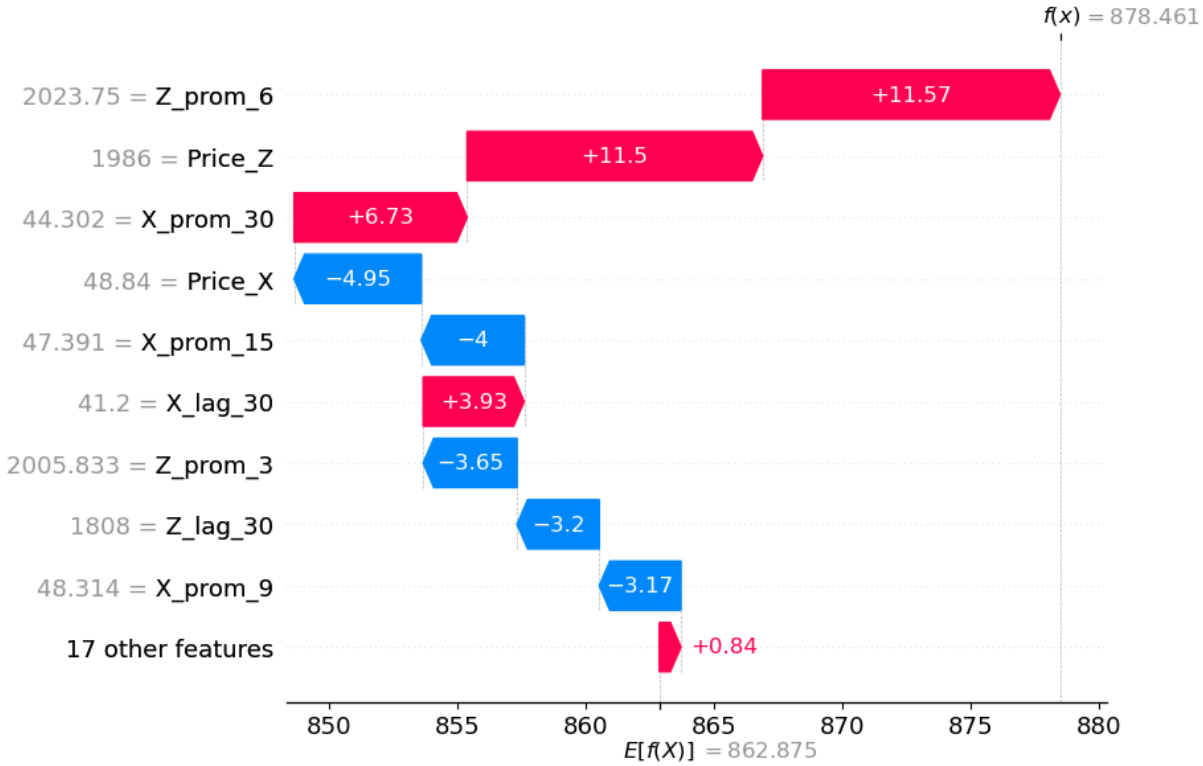
```
importances = model2.feature_importances_
importance_df = pd.DataFrame({
    'Feature': features,
    'Importance': importances
}).sort_values(by='Importance', ascending=False).head(20)

print("\n🎯 Feature Importance (XGBoost):\n", importance_df)

# Graficar
plt.figure(figsize=(10, 6))
plt.barh(importance_df['Feature'], importance_df['Importance'])
plt.gca().invert_yaxis()
plt.title("XGBoost Feature Importance")
plt.show()
```

```
🎯 Feature Importance (XGBoost):
      Feature  Importance
5    Z_prom_6    0.448027
1     Price_Z    0.324698
3    Z_prom_3    0.091025
20    Z_lag_1    0.037656
7    Z_prom_9    0.016213
21    Z_lag_3    0.013812
10  X_prom_15    0.012653
4    X_prom_6    0.010207
6    X_prom_9    0.009821
13  Z_prom_30    0.008227
12  X_prom_30    0.005405
25   Z_lag_30    0.002855
19   X_lag_30    0.002584
17   X_lag_12    0.002111
14    X_lag_1    0.001778
22    Z_lag_6    0.001618
8   X_prom_12    0.001600
9   Z_prom_12    0.001583
11  Z_prom_15    0.001518
18   X_lag_15    0.001287
```



In [89]:
```
# ------------------------------
# 5. SHAP values
# ------------------------------
explainer = shap.Explainer(model2, X_train)
shap_values = explainer(X_test)

# Resumen global
shap.summary_plot(shap_values, X_test)

# También puedes ver un resumen por instancia individual:
shap.plots.waterfall(shap_values[0])
```


Price_Z
Z_prom_6
Z_prom_3
Z_lag_1
Price_X
Z_lag_3
Z_lag_6
Z_lag_30
Z_prom_15
X_prom_9
X_prom_15
Z_prom_30
Z_prom_9
X_lag_30
X_lag_12
Z_lag_15
X_lag_6
X_prom_3
X_lag_3
X_lag_1
0
50
100
150
200
SHAP value (impact on model output)
High
Feature value
Low



In [90]:

```
import numpy as np
import pandas as pd

# Convertir a array
shap_vals = shap_values.values

# Importancia global (media absoluta)
shap_importance = np.abs(shap_vals).mean(axis=0)

# Crear DataFrame
feature_importance = pd.DataFrame({
    "Feature": X_test.columns,
    "Importance": shap_importance
})

# Ordenar
feature_importance = feature_importance.sort_values(by="Importance", ascending=Fals

# Top 10
top10_model2 = feature_importance.head(10)

print(top10_model2)
```

```
      Feature  Importance
1     Price_Z  117.334269
5    Z_prom_6   16.908375
3    Z_prom_3   15.054872
20    Z_lag_1    7.816849
0     Price_X    6.458108
21    Z_lag_3    4.939049
22    Z_lag_6    3.777287
25   Z_lag_30    3.722854
11  Z_prom_15    3.564239
6    X_prom_9    3.019787
```

Se evaluaron modelos con selección de variables basada en SHAP; sin embargo, se observó una disminución en la precisión predictiva. Por ello, se optó por utilizar el modelo completo, el cual captura de mejor manera las relaciones no lineales entre las variables, logrando mejores métricas de desempeño.

Entrenar el modelo con el 100%

In [91]:

```python
from xgboost import XGBRegressor

# Dataset limpio
df_ml = df

# Features (las mismas que usaste en el modelo final)
X = df_ml[features]

y1 = df_ml[target1]
y2 = df_ml[target2]

# Modelos finales
model1_final = XGBRegressor(random_state=42)
model2_final = XGBRegressor(random_state=42)

# Entrenamiento con TODO
model1_final.fit(X, y1)
model2_final.fit(X, y2)
```

Out[91]:

```
XGBRegressor
XGBRegressor(base_score=None, booster=None, callbacks=None,
             colsample_bylevel=None, colsample_bynode=None,
             colsample_bytree=None, device=None, early_stopping_rounds=
None,
             enable_categorical=False, eval_metric=None, feature_types=
None,
             feature_weights=None, gamma=None, grow_policy=None,
             importance_type=None, interaction_constraints=None,
             learning_rate=None, max_bin=None, max_cat_threshold=None,
```

Forecast

En esta seccion encontraremos los siguientes pronosticos

- Price X y Price Z
- Medias Moviles
- Variables rezagadas
- Todo esto con el fin de alimentar el modelo XGB y pronosticar los equipos 1 y 2

In [96]:
```
equipos
```

Out[96]:

	Date	Price_X	Price_Y	Price_Z	Price_Equipo1	Price_Equipo2
0	2010-01-04	80.12	527.50	2225.25	434.73	931.73
1	2010-01-05	80.59	527.50	2246.50	449.97	968.56
2	2010-01-06	81.89	527.50	2302.50	444.48	960.51
3	2010-01-07	81.51	527.50	2306.50	440.90	960.14
4	2010-01-08	81.37	552.50	2261.25	448.82	949.55
...	...	...	...	...	...	...
3525	2023-08-25	84.48	554.00	2123.50	450.36	899.65
3526	2023-08-28	84.42	549.00	2116.25	469.03	933.20
3527	2023-08-29	85.49	545.33	2116.25	460.42	901.79
3528	2023-08-30	85.86	543.67	2140.25	464.80	945.40
3529	2023-08-31	86.86	545.00	2165.25	451.73	955.35

3530 rows × 6 columns

In [97]:
```
print(df.reset_index()["Date"].max() ,df.reset_index()["Date"].min())
```

```
2023-08-31 00:00:00 2010-01-04 00:00:00
```

In [98]:
```
last_date = df.reset_index()["Date"].max()
```

In [99]:
```
last_date
```

Out[99]:
```
Timestamp('2023-08-31 00:00:00')
```

Pronosticamos 30 dias en el futuro

In [103…
```
future_dates = pd.date_range(start=last_date, periods=30, freq='D')[1:]
```

In [104…
```
forecast_df = pd.DataFrame({
    "Date": future_dates
```

```
}).reset_index(drop = True)
```

In [105…

```
df_real_fsct = pd.concat([equipos.drop(["Price_Y"],axis=1),forecast_df], axis=0)
```

Modelos Sarima

Gracias al anlisis del modelo SARIMAX en pasos previos, logramos hallar los parametros que mejor se adaptan a las materias primas, de esta manera logramos pronosticarlo

In [106…

```
model_X = SARIMAX(df["Price_X"], order=(1,1,1), seasonal_order=(1,1,1,12))
results_X = model_X.fit()
model_Z = SARIMAX(df["Price_Z"], order=(1,1,1), seasonal_order=(1,1,1,12))
results_Z = model_Z.fit()
```

```
c:\Users\Sebastian.garzon.LAPTOP-G9J39UDS\anaconda3\Lib\site-packages\statsmodels\ts
a\base\tsa_model.py:473: ValueWarning: A date index has been provided, but it has no
associated frequency information and so will be ignored when e.g. forecasting.
  self._init_dates(dates, freq)
c:\Users\Sebastian.garzon.LAPTOP-G9J39UDS\anaconda3\Lib\site-packages\statsmodels\ts
a\base\tsa_model.py:473: ValueWarning: A date index has been provided, but it has no
associated frequency information and so will be ignored when e.g. forecasting.
  self._init_dates(dates, freq)
c:\Users\Sebastian.garzon.LAPTOP-G9J39UDS\anaconda3\Lib\site-packages\statsmodels\ts
a\statespace\sarimax.py:966: UserWarning: Non-stationary starting autoregressive par
ameters found. Using zeros as starting parameters.
  warn('Non-stationary starting autoregressive parameters'
c:\Users\Sebastian.garzon.LAPTOP-G9J39UDS\anaconda3\Lib\site-packages\statsmodels\ts
a\statespace\sarimax.py:978: UserWarning: Non-invertible starting MA parameters foun
d. Using zeros as starting parameters.
  warn('Non-invertible starting MA parameters found.'
c:\Users\Sebastian.garzon.LAPTOP-G9J39UDS\anaconda3\Lib\site-packages\statsmodels\ts
a\base\tsa_model.py:473: ValueWarning: A date index has been provided, but it has no
associated frequency information and so will be ignored when e.g. forecasting.
  self._init_dates(dates, freq)
c:\Users\Sebastian.garzon.LAPTOP-G9J39UDS\anaconda3\Lib\site-packages\statsmodels\ts
a\base\tsa_model.py:473: ValueWarning: A date index has been provided, but it has no
associated frequency information and so will be ignored when e.g. forecasting.
  self._init_dates(dates, freq)
```

In [107…

```
steps = 30

forecast_X = results_X.get_forecast(steps=steps)
forecast_Z = results_Z.get_forecast(steps=steps)

X_pred = forecast_X.predicted_mean.reset_index(drop = True)
Z_pred = forecast_Z.predicted_mean.reset_index(drop = True)
```

```
c:\Users\Sebastian.garzon.LAPTOP-G9J39UDS\anaconda3\Lib\site-packages\statsmodels\ts
a\base\tsa_model.py:837: ValueWarning: No supported index is available. Prediction r
esults will be given with an integer index beginning at `start`.
  return get_prediction_index(
c:\Users\Sebastian.garzon.LAPTOP-G9J39UDS\anaconda3\Lib\site-packages\statsmodels\ts
a\base\tsa_model.py:837: FutureWarning: No supported index is available. In the next
version, calling this method in a model without a supported index will result in an
exception.
  return get_prediction_index(
c:\Users\Sebastian.garzon.LAPTOP-G9J39UDS\anaconda3\Lib\site-packages\statsmodels\ts
a\base\tsa_model.py:837: ValueWarning: No supported index is available. Prediction r
esults will be given with an integer index beginning at `start`.
  return get_prediction_index(
```

In [108…

```
conf_X = forecast_X.conf_int().reset_index(drop = True)
conf_Z = forecast_Z.conf_int().reset_index(drop = True)
```

In [109…

```
pred_x_conso = pd.concat([forecast_df,conf_X,X_pred], axis = 1).reset_index(drop =
pred_z_conso = pd.concat([forecast_df,conf_Z,Z_pred], axis = 1).reset_index(drop =
```

In [110…

```
df_x_forecast = pd.DataFrame({
    "Date": forecast_df["Date"],
    "X_mean": pred_x_conso["predicted_mean"],
    "X_lower": pred_x_conso["lower Price_X"],
    "X_upper": pred_x_conso["upper Price_X"]
})

df_z_forecast = pd.DataFrame({
    "Date": forecast_df["Date"],
    "Z_mean": pred_z_conso["predicted_mean"],
    "Z_lower": pred_z_conso["lower Price_Z"],
    "Z_upper": pred_z_conso["upper Price_Z"]
})
```

In [111…

```
df
```

Out[111…

Date	Price_X	Price_Y	Price_Z	Price_Equipo1	Price_Equipo2	X_diff	Z_diff	X_prom_3	
2010-01-04	80.12	527.50	2225.25	434.73	931.73	NaN	NaN	NaN	
2010-01-05	80.59	527.50	2246.50	449.97	968.56	0.47	21.25	NaN	
2010-01-06	81.89	527.50	2302.50	444.48	960.51	1.30	56.00	80.866667	2
2010-01-07	81.51	527.50	2306.50	440.90	960.14	-0.38	4.00	81.330000	2
2010-01-08	81.37	552.50	2261.25	448.82	949.55	-0.14	-45.25	81.590000	2
...	...	...	...	...	...	...	...	...	
2023-08-25	84.48	554.00	2123.50	450.36	899.65	1.12	4.25	83.683333	2
2023-08-28	84.42	549.00	2116.25	469.03	933.20	-0.06	-7.25	84.086667	2
2023-08-29	85.49	545.33	2116.25	460.42	901.79	1.07	0.00	84.796667	2
2023-08-30	85.86	543.67	2140.25	464.80	945.40	0.37	24.00	85.256667	2
2023-08-31	86.86	545.00	2165.25	451.73	955.35	1.00	25.00	86.070000	2

3530 rows × 31 columns

Pronostico Price X

In [112…

```
import matplotlib.pyplot as plt

plt.figure(figsize=(12,5))

# Real
plt.plot(df.reset_index()["Date"].iloc[-60:],
         df["Price_X"].iloc[-60:],
         label="Real", linewidth=2)

# Forecast
plt.plot(df_x_forecast["Date"],
         df_x_forecast["X_mean"],
         linestyle="--", label="Pronóstico")

# Intervalo
```

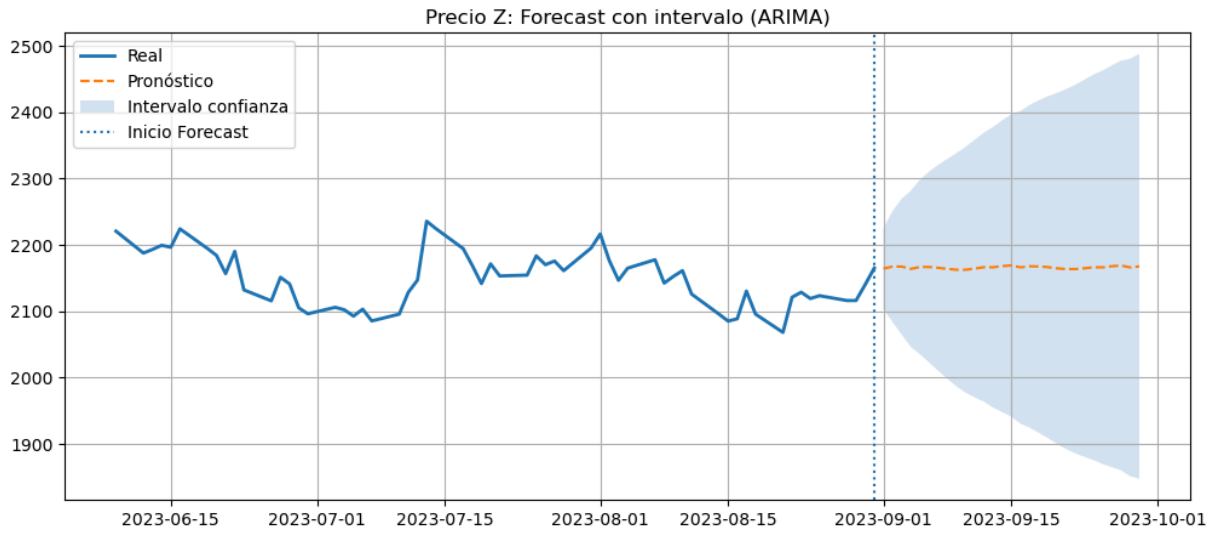
```
plt.fill_between(df_x_forecast["Date"],
                 df_x_forecast["X_lower"],
                 df_x_forecast["X_upper"],
                 alpha=0.2, label="Intervalo confianza")

# Línea de corte
plt.axvline(x=df.reset_index()["Date"].iloc[-1], linestyle=":", label="Inicio Forec

plt.title("Precio X: Forecast con intervalo (ARIMA)")
plt.legend()
plt.grid()

plt.show()
```

Pronostico Price Z

In [113…

```
plt.figure(figsize=(12,5))

# Real
plt.plot(df.reset_index()["Date"].iloc[-60:],
         df["Price_Z"].iloc[-60:],
         label="Real", linewidth=2)

# Forecast
plt.plot(df_z_forecast["Date"],
         df_z_forecast["Z_mean"],
         linestyle="--", label="Pronóstico")

# Intervalo
plt.fill_between(df_z_forecast["Date"],
                 df_z_forecast["Z_lower"],
                 df_z_forecast["Z_upper"],
                 alpha=0.2, label="Intervalo confianza")

# Línea de corte
plt.axvline(x=df.reset_index()["Date"].iloc[-1], linestyle=":", label="Inicio Forec

plt.title("Precio Z: Forecast con intervalo (ARIMA)")
```

```
plt.legend()
plt.grid()

plt.show()
```



In [114…

```
pred_conso = pred_x_conso.rename(columns={"predicted_mean": "Price_X"})[["Date","Pr
    pred_z_conso.rename(columns={"predicted_mean": "Price_Z"})[["Date","Price_Z"]],
)
```

In [115…

```
pred_conso
```

Out[115…

	Date	Price_X	Price_Z
0	2023-09-01	86.502163	2164.256648
1	2023-09-02	86.503417	2167.420848
2	2023-09-03	86.590910	2167.368206
3	2023-09-04	86.463308	2163.956558
4	2023-09-05	86.436772	2166.770162
5	2023-09-06	86.224095	2166.772924
6	2023-09-07	86.354412	2165.521268
7	2023-09-08	86.427628	2163.887930
8	2023-09-09	86.605362	2162.618418
9	2023-09-10	86.624109	2162.618968
10	2023-09-11	86.745775	2164.391392
11	2023-09-12	86.766390	2166.506477
12	2023-09-13	86.493406	2166.349166
13	2023-09-14	86.504732	2168.036684
14	2023-09-15	86.596679	2169.123602
15	2023-09-16	86.496459	2166.199417
16	2023-09-17	86.499458	2167.794676
17	2023-09-18	86.320104	2167.571373
18	2023-09-19	86.458198	2166.548420
19	2023-09-20	86.499764	2164.755450
20	2023-09-21	86.690998	2163.643674
21	2023-09-22	86.675182	2163.641854
22	2023-09-23	86.790590	2164.846485
23	2023-09-24	86.777683	2166.375094
24	2023-09-25	86.509371	2166.239508
25	2023-09-26	86.522262	2167.889058
26	2023-09-27	86.615277	2169.005233
27	2023-09-28	86.516738	2166.093484
28	2023-09-29	86.521329	2167.657576
29	NaT	86.343578	2167.428474

In [116…
```
df_real_fsct = pd.concat([equipos.reset_index(drop = True),pred_conso
],axis = 0).drop(["Price_Y","Price_Equipo1","Price_Equipo2"], axis = 1)
```

In [117…
```
df_real_fsct["Date"] = pd.to_datetime(df_real_fsct["Date"])
```

In [118…
```
df_real_fsct
```

Out[118…

	Date	Price_X	Price_Z
0	2010-01-04	80.120000	2225.250000
1	2010-01-05	80.590000	2246.500000
2	2010-01-06	81.890000	2302.500000
3	2010-01-07	81.510000	2306.500000
4	2010-01-08	81.370000	2261.250000
...	...	...	...
25	2023-09-26	86.522262	2167.889058
26	2023-09-27	86.615277	2169.005233
27	2023-09-28	86.516738	2166.093484
28	2023-09-29	86.521329	2167.657576
29	NaT	86.343578	2167.428474

3560 rows × 3 columns

Promedio Movil

In [119…
```
df_real_fsct["X_prom_3"] = df_real_fsct["Price_X"].rolling(window=3).mean()
df_real_fsct["Z_prom_3"] = df_real_fsct["Price_Z"].rolling(window=3).mean()
df_real_fsct["X_prom_6"] = df_real_fsct["Price_X"].rolling(window=6).mean()
df_real_fsct["Z_prom_6"] = df_real_fsct["Price_Z"].rolling(window=6).mean()
df_real_fsct["X_prom_9"] = df_real_fsct["Price_X"].rolling(window=9).mean()
df_real_fsct["Z_prom_9"] = df_real_fsct["Price_Z"].rolling(window=9).mean()
df_real_fsct["X_prom_12"] = df_real_fsct["Price_X"].rolling(window=12).mean()
df_real_fsct["Z_prom_12"] = df_real_fsct["Price_Z"].rolling(window=12).mean()
df_real_fsct["X_prom_15"] = df_real_fsct["Price_X"].rolling(window=15).mean()
df_real_fsct["Z_prom_15"] = df_real_fsct["Price_Z"].rolling(window=15).mean()
df_real_fsct["X_prom_30"] = df_real_fsct["Price_X"].rolling(window=30).mean()
df_real_fsct["Z_prom_30"] = df_real_fsct["Price_Z"].rolling(window=30).mean()
```

In [120…
```
df_real_fsct
```

Out[120…

	Date	Price_X	Price_Z	X_prom_3	Z_prom_3	X_prom_6	Z_prom_6	X_pron
0	2010-01-04	80.120000	2225.250000	NaN	NaN	NaN	NaN	N
1	2010-01-05	80.590000	2246.500000	NaN	NaN	NaN	NaN	N
2	2010-01-06	81.890000	2302.500000	80.866667	2258.083333	NaN	NaN	N
3	2010-01-07	81.510000	2306.500000	81.330000	2285.166667	NaN	NaN	N
4	2010-01-08	81.370000	2261.250000	81.590000	2290.083333	NaN	NaN	N
...	...	...	...	...	...	...	...	
25	2023-09-26	86.522262	2167.889058	86.603105	2166.834553	86.661015	2165.439279	86.5826
26	2023-09-27	86.615277	2169.005233	86.548970	2167.711266	86.648394	2166.332872	86.6154
27	2023-09-28	86.516738	2166.093484	86.551426	2167.662592	86.621987	2166.741477	86.6219
28	2023-09-29	86.521329	2167.657576	86.551115	2167.585431	86.577110	2167.209992	86.6243
29	NaT	86.343578	2167.428474	86.460548	2167.059844	86.504759	2167.385555	86.5857

3560 rows × 15 columns

Precios Rezagados

In [121…

```
# Lags reales (valores pasados)
df_real_fsct["X_lag_1"] = df_real_fsct["Price_X"].shift(1)
df_real_fsct["X_lag_3"] = df_real_fsct["Price_X"].shift(3)
df_real_fsct["X_lag_6"] = df_real_fsct["Price_X"].shift(6)
df_real_fsct["X_lag_12"] = df_real_fsct["Price_X"].shift(12)
df_real_fsct["X_lag_15"] = df_real_fsct["Price_X"].shift(15)
df_real_fsct["X_lag_30"] = df_real_fsct["Price_X"].shift(30)
df_real_fsct["Z_lag_1"] = df_real_fsct["Price_Z"].shift(1)
df_real_fsct["Z_lag_3"] = df_real_fsct["Price_Z"].shift(3)
df_real_fsct["Z_lag_6"] = df_real_fsct["Price_Z"].shift(6)
df_real_fsct["Z_lag_12"] = df_real_fsct["Price_Z"].shift(12)
df_real_fsct["Z_lag_15"] = df_real_fsct["Price_Z"].shift(15)
df_real_fsct["Z_lag_30"] = df_real_fsct["Price_Z"].shift(30)
```

In [122…

```
df_real_fsct = df_real_fsct.reset_index(drop=True)
```

In [123…

```
df_real_fsct
```

Out[123…

	Date	Price_X	Price_Z	X_prom_3	Z_prom_3	X_prom_6	Z_prom_6	X_pr
0	2010-01-04	80.120000	2225.250000	NaN	NaN	NaN	NaN	
1	2010-01-05	80.590000	2246.500000	NaN	NaN	NaN	NaN	
2	2010-01-06	81.890000	2302.500000	80.866667	2258.083333	NaN	NaN	
3	2010-01-07	81.510000	2306.500000	81.330000	2285.166667	NaN	NaN	
4	2010-01-08	81.370000	2261.250000	81.590000	2290.083333	NaN	NaN	
...	...	...	...	...	...	...	...	
3555	2023-09-26	86.522262	2167.889058	86.603105	2166.834553	86.661015	2165.439279	86.58
3556	2023-09-27	86.615277	2169.005233	86.548970	2167.711266	86.648394	2166.332872	86.6
3557	2023-09-28	86.516738	2166.093484	86.551426	2167.662592	86.621987	2166.741477	86.62
3558	2023-09-29	86.521329	2167.657576	86.551115	2167.585431	86.577110	2167.209992	86.62
3559	NaT	86.343578	2167.428474	86.460548	2167.059844	86.504759	2167.385555	86.58

3560 rows × 27 columns

In [124…

```
df_feat_fsct = df_real_fsct[df_real_fsct["Date"] > pd.Timestamp(last_date) ]
```

In [125…

```
df_feat_fsct
```

Out[125...

	Date	Price_X	Price_Z	X_prom_3	Z_prom_3	X_prom_6	Z_prom_6	X_pr
3530	2023-09-01	86.502163	2164.256648	86.407388	2156.585549	85.602027	2137.626108	84.9
3531	2023-09-02	86.503417	2167.420848	86.621860	2165.642498	85.939263	2144.946249	85.1
3532	2023-09-03	86.590910	2167.368206	86.532163	2166.348567	86.301082	2153.465950	85.5
3533	2023-09-04	86.463308	2163.956558	86.519212	2166.248537	86.463300	2161.417043	85.9
3534	2023-09-05	86.436772	2166.770162	86.496997	2166.031642	86.559428	2165.837070	86.1
3535	2023-09-06	86.224095	2166.772924	86.374725	2165.833214	86.453444	2166.090891	86.3
3536	2023-09-07	86.354412	2165.521268	86.338426	2166.354784	86.428819	2166.301661	86.4
3537	2023-09-08	86.427628	2163.887930	86.335378	2165.394040	86.416188	2165.712841	86.4
3538	2023-09-09	86.605362	2162.618418	86.462467	2164.009205	86.418596	2164.921210	86.4
3539	2023-09-10	86.624109	2162.618968	86.552366	2163.041772	86.445396	2164.698278	86.4
3540	2023-09-11	86.745775	2164.391392	86.658415	2163.209593	86.496897	2164.301817	86.4
3541	2023-09-12	86.766390	2166.506477	86.712091	2164.505612	86.587279	2164.257409	86.5
3542	2023-09-13	86.493406	2166.349166	86.668524	2165.749012	86.610445	2164.395392	86.5
3543	2023-09-14	86.504732	2168.036684	86.588176	2166.964109	86.623296	2165.086851	86.5
3544	2023-09-15	86.596679	2169.123602	86.531606	2167.836484	86.621849	2166.171048	86.5
3545	2023-09-16	86.496459	2166.199417	86.532623	2167.786568	86.600574	2166.767790	86.5
3546	2023-09-17	86.499458	2167.794676	86.530865	2167.705898	86.559521	2167.335004	86.5
3547	2023-09-18	86.320104	2167.571373	86.438673	2167.188489	86.485140	2167.512486	86.5
3548	2023-09-19	86.458198	2166.548420	86.425920	2167.304823	86.479272	2167.545695	86.5

	Date	Price_X	Price_Z	X_prom_3	Z_prom_3	X_prom_6	Z_prom_6	X_pr
3549	2023-09-20	86.499764	2164.755450	86.426022	2166.291748	86.478444	2166.998823	86.5
3550	2023-09-21	86.690998	2163.643674	86.549654	2164.982515	86.494164	2166.085502	86.5
3551	2023-09-22	86.675182	2163.641854	86.621982	2164.013659	86.523951	2165.659241	86.5
3552	2023-09-23	86.790590	2164.846485	86.718924	2164.044004	86.572473	2165.167876	86.5
3553	2023-09-24	86.777683	2166.375094	86.747819	2164.954478	86.648736	2164.968496	86.5
3554	2023-09-25	86.509371	2166.239508	86.692548	2165.820362	86.657265	2164.917011	86.5
3555	2023-09-26	86.522262	2167.889058	86.603105	2166.834553	86.661015	2165.439279	86.5
3556	2023-09-27	86.615277	2169.005233	86.548970	2167.711266	86.648394	2166.332872	86.6
3557	2023-09-28	86.516738	2166.093484	86.551426	2167.662592	86.621987	2166.741477	86.6
3558	2023-09-29	86.521329	2167.657576	86.551115	2167.585431	86.577110	2167.209992	86.6

29 rows × 27 columns

Pronostico Equipo 1 y Equipo 2

Se realiza el pronostico de los equipos 1 y 2 con el modelo previamente entrenado

In [128…

```
df_pred_F = pd.DataFrame({
    "Date": df_feat_fsct["Date"],
    "Price_Equipo1_pred": model1_final.predict(df_feat_fsct.drop("Date", axis=1)),
    "Price_Equipo2_pred": model2_final.predict(df_feat_fsct.drop("Date", axis=1))
})
```

In [129…

```
df_pred_F
```

Out[129…

	Date	Price_Equipo1_pred	Price_Equipo2_pred
3530	2023-09-01	468.931335	954.877869
3531	2023-09-02	470.946045	933.095398
3532	2023-09-03	472.459564	933.094055
3533	2023-09-04	465.399811	931.915100
3534	2023-09-05	465.087189	934.483276
3535	2023-09-06	491.099792	933.500671
3536	2023-09-07	487.210785	928.940735
3537	2023-09-08	489.326447	928.558594
3538	2023-09-09	483.367249	928.465515
3539	2023-09-10	486.009888	926.782410
3540	2023-09-11	486.009888	932.240417
3541	2023-09-12	487.523407	933.924316
3542	2023-09-13	491.556122	935.700500
3543	2023-09-14	478.723877	935.700500
3544	2023-09-15	479.405243	939.994690
3545	2023-09-16	479.405243	939.994690
3546	2023-09-17	488.196014	939.994690
3547	2023-09-18	485.042542	939.994690
3548	2023-09-19	481.068207	938.776611
3549	2023-09-20	485.109680	944.824524
3550	2023-09-21	484.832520	937.925964
3551	2023-09-22	481.802521	944.880188
3552	2023-09-23	484.512299	943.048340
3553	2023-09-24	487.134613	937.000427
3554	2023-09-25	487.134613	938.776611
3555	2023-09-26	487.134613	939.994690
3556	2023-09-27	488.937653	938.218506
3557	2023-09-28	491.580292	939.994690
3558	2023-09-29	491.580292	939.994690

In [130…

```
df.reset_index()
```

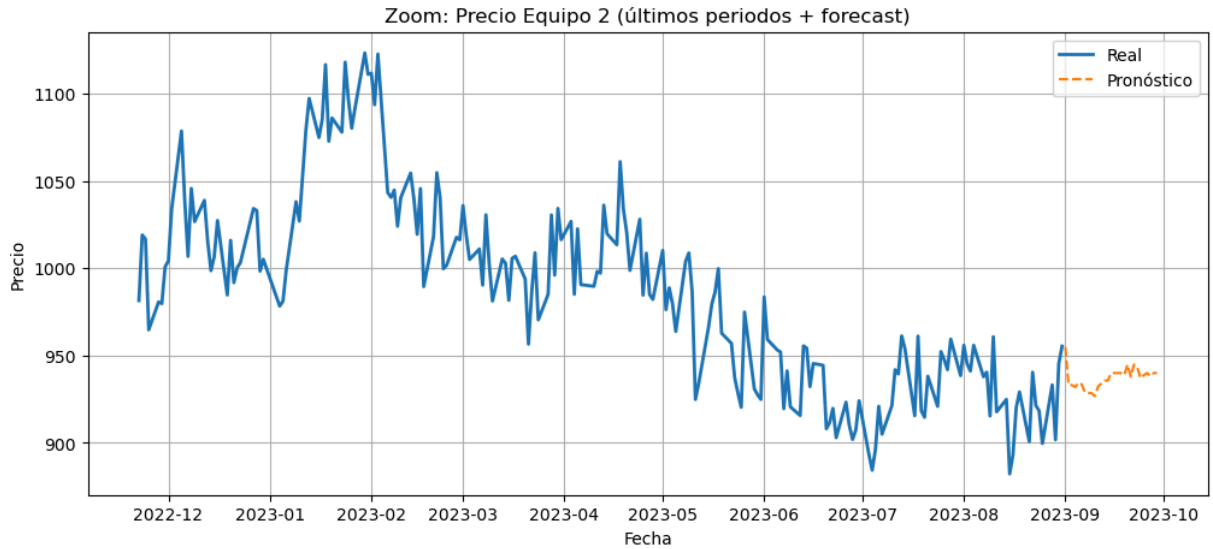
Out[130…

	Date	Price_X	Price_Y	Price_Z	Price_Equipo1	Price_Equipo2	X_diff	Z_diff	X_pro
0	2010-01-04	80.12	527.50	2225.25	434.73	931.73	NaN	NaN	I
1	2010-01-05	80.59	527.50	2246.50	449.97	968.56	0.47	21.25	I
2	2010-01-06	81.89	527.50	2302.50	444.48	960.51	1.30	56.00	80.866
3	2010-01-07	81.51	527.50	2306.50	440.90	960.14	-0.38	4.00	81.330
4	2010-01-08	81.37	552.50	2261.25	448.82	949.55	-0.14	-45.25	81.590
...	...	...	...	...	...	...	...	...	
3525	2023-08-25	84.48	554.00	2123.50	450.36	899.65	1.12	4.25	83.683
3526	2023-08-28	84.42	549.00	2116.25	469.03	933.20	-0.06	-7.25	84.086
3527	2023-08-29	85.49	545.33	2116.25	460.42	901.79	1.07	0.00	84.796
3528	2023-08-30	85.86	543.67	2140.25	464.80	945.40	0.37	24.00	85.256
3529	2023-08-31	86.86	545.00	2165.25	451.73	955.35	1.00	25.00	86.070

3530 rows × 32 columns

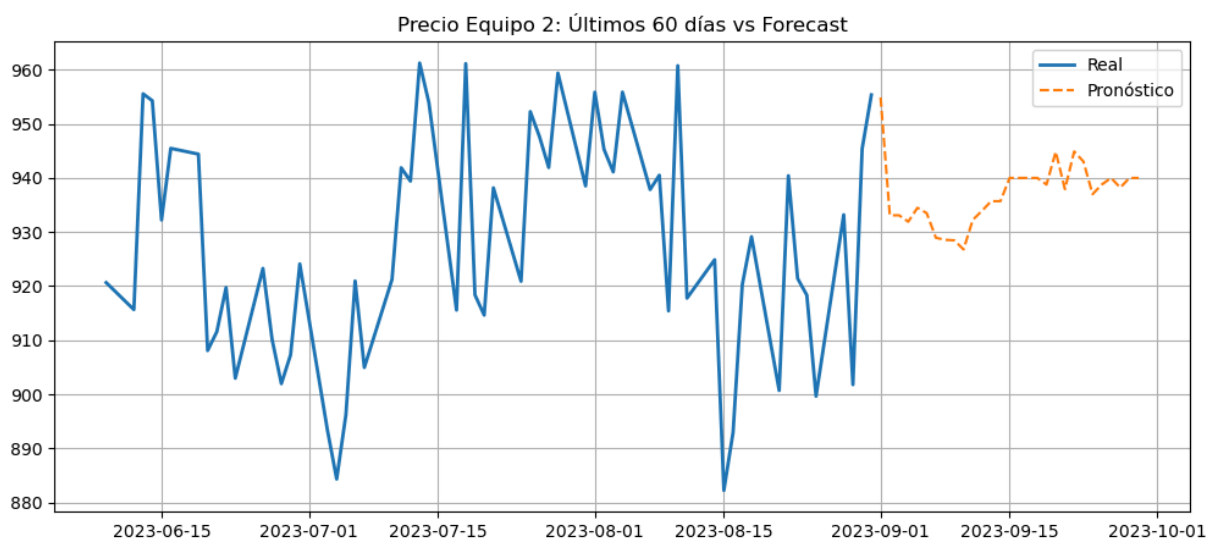
In [131…

```
import matplotlib.pyplot as plt

plt.figure(figsize=(12,5))

# Últimos 200 puntos históricos
plt.plot(df.reset_index()["Date"].iloc[-200:], df["Price_Equipo2"].iloc[-200:],
         label="Real", linewidth=2)

# Forecast
plt.plot(df_pred_F["Date"], df_pred_F["Price_Equipo2_pred"],
         label="Pronóstico", linestyle="--")

plt.title("Zoom: Precio Equipo 2 (últimos periodos + forecast)")
plt.xlabel("Fecha")
plt.ylabel("Precio")
plt.legend()
plt.grid()

plt.show()
```



In [132…

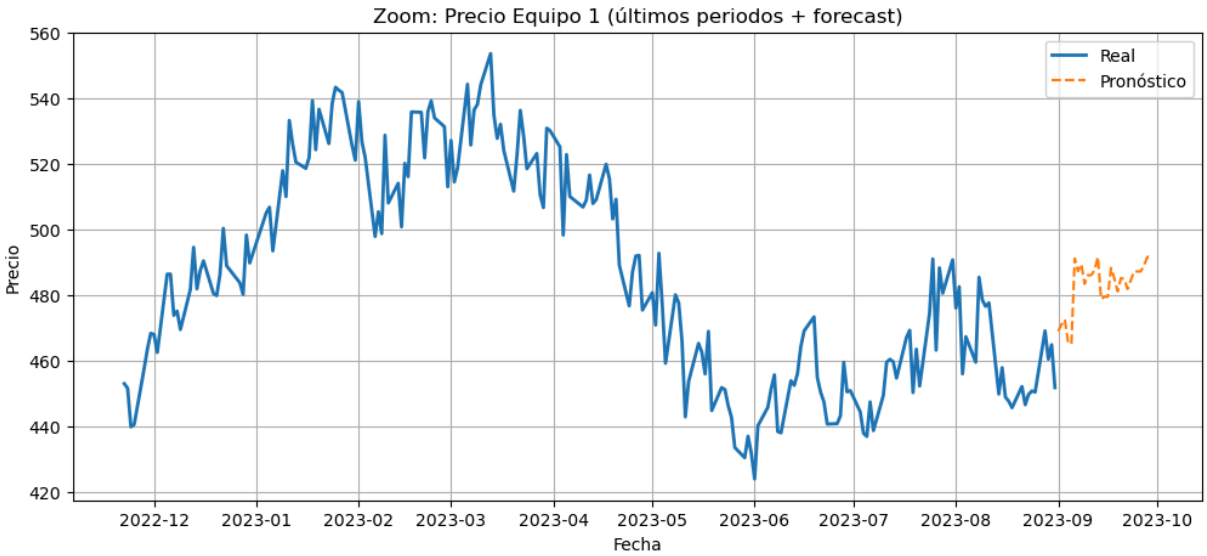
```
plt.figure(figsize=(12,5))

# Últimos 60 días
plt.plot(df.reset_index()["Date"].iloc[-60:], df["Price_Equipo2"].iloc[-60:],
         label="Real", linewidth=2)

# Forecast
plt.plot(df_pred_F["Date"], df_pred_F["Price_Equipo2_pred"],
         label="Pronóstico", linestyle="--")

plt.title("Precio Equipo 2: Últimos 60 días vs Forecast")
plt.legend()
plt.grid()

plt.show()
```



In [133…

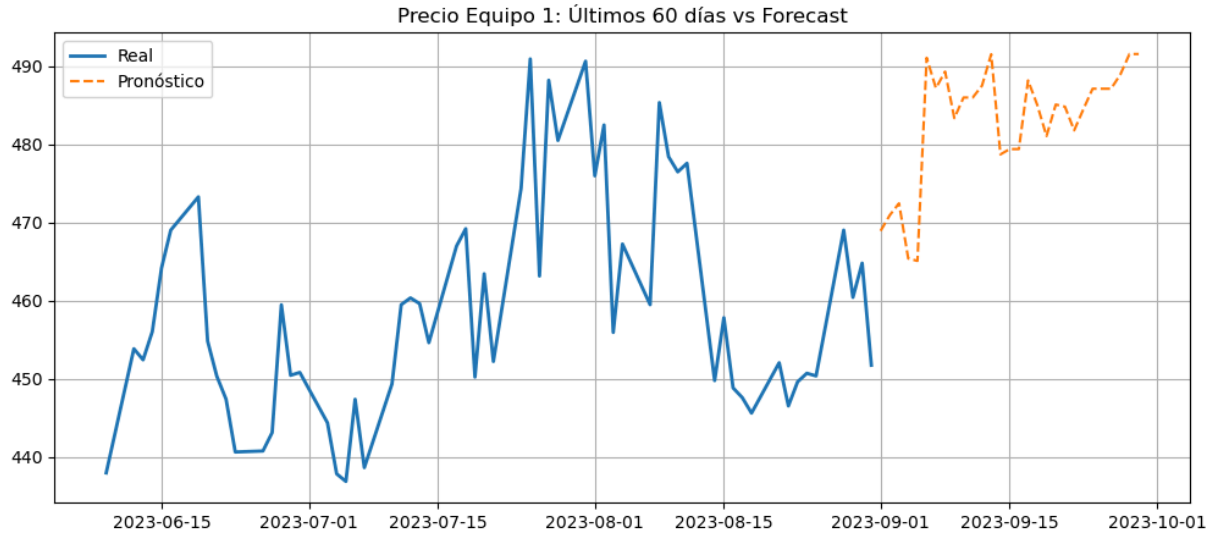
```
import matplotlib.pyplot as plt

plt.figure(figsize=(12,5))

# Últimos 200 puntos históricos
```

```
plt.plot(df.reset_index()["Date"].iloc[-200:], df["Price_Equipo1"].iloc[-200:],
         label="Real", linewidth=2)

# Forecast
plt.plot(df_pred_F["Date"], df_pred_F["Price_Equipo1_pred"],
         label="Pronóstico", linestyle="--")

plt.title("Zoom: Precio Equipo 1 (últimos periodos + forecast)")
plt.xlabel("Fecha")
plt.ylabel("Precio")
plt.legend()
plt.grid()

plt.show()
```



In [134…

```
plt.figure(figsize=(12,5))

# Últimos 60 días
plt.plot(df.reset_index()["Date"].iloc[-60:], df["Price_Equipo1"].iloc[-60:],
         label="Real", linewidth=2)

# Forecast
plt.plot(df_pred_F["Date"], df_pred_F["Price_Equipo1_pred"],
         label="Pronóstico", linestyle="--")

plt.title("Precio Equipo 1: Últimos 60 días vs Forecast")
plt.legend()
plt.grid()

plt.show()
```



Pronostico de los Intervalos

Se proyectan los intervalos Lower y Upper para que el usuario tenga un rango para estimar como se puede mover los precios de los equipos 1 y 2

In [135…

```
# LOWER
pred_conso_lower = pred_x_conso.rename(columns={"lower Price_X": "Price_X"})[["Date
    pred_z_conso.rename(columns={"lower Price_Z": "Price_Z"})[["Date","Price_Z"]],
    on="Date", how="left"
)

# UPPER
pred_conso_upper = pred_x_conso.rename(columns={"upper Price_X": "Price_X"})[["Date
    pred_z_conso.rename(columns={"upper Price_Z": "Price_Z"})[["Date","Price_Z"]],
    on="Date", how="left"
)
```

In [136…

```
def crear_features(df_base):
    df_temp = df_base.copy()

    # Promedios móviles
    for w in [3,6,9,12,15,30]:
        df_temp[f"X_prom_{w}"] = df_temp["Price_X"].rolling(window=w).mean()
        df_temp[f"Z_prom_{w}"] = df_temp["Price_Z"].rolling(window=w).mean()

    # Lags
    for lag in [1,3,6,12,15,30]:
        df_temp[f"X_lag_{lag}"] = df_temp["Price_X"].shift(lag)
        df_temp[f"Z_lag_{lag}"] = df_temp["Price_Z"].shift(lag)

    return df_temp
```

In [137…

```
# LOWER
df_lower = pd.concat([equipos.reset_index(drop=True), pred_conso_lower], axis=0)
df_lower = df_lower.drop(["Price_Y","Price_Equipo1","Price_Equipo2"], axis=1)
df_lower["Date"] = pd.to_datetime(df_lower["Date"])
df_lower = crear_features(df_lower).reset_index(drop=True)
```

```
# UPPER
df_upper = pd.concat([equipos.reset_index(drop=True), pred_conso_upper], axis=0)
df_upper = df_upper.drop(["Price_Y","Price_Equipo1","Price_Equipo2"], axis=1)
df_upper["Date"] = pd.to_datetime(df_upper["Date"])
df_upper = crear_features(df_upper).reset_index(drop=True)
```

In [138…

```
df_lower_fsct = df_lower[df_lower["Date"] > pd.Timestamp(last_date)]
df_upper_fsct = df_upper[df_upper["Date"] > pd.Timestamp(last_date)]
```

In [139…

```
df_lower_fsct
```

Out[139...

	Date	Price_X	Price_Z	X_prom_3	Z_prom_3	X_prom_6	Z_prom_6	X_pr
3530	2023-09-01	83.435610	2103.240578	85.385203	2136.246859	85.090935	2127.456763	84.5
3531	2023-09-02	82.219386	2083.966230	84.171666	2117.485603	84.714166	2120.867801	84.3
3532	2023-09-03	81.398040	2065.144857	82.351012	2084.117222	84.210506	2112.350278	84.1
3533	2023-09-04	80.519488	2046.447312	81.378971	2065.186133	83.382087	2100.716496	83.8
3534	2023-09-05	79.840891	2035.500289	80.586140	2049.030819	82.378903	2083.258211	83.3
3535	2023-09-06	79.044770	2023.169723	79.801716	2035.039108	81.076364	2059.578165	82.7
3536	2023-09-07	78.642404	2010.508190	79.176022	2023.059400	80.277496	2044.122767	81.9
3537	2023-09-08	78.222221	1998.275592	78.636465	2010.651168	79.611302	2029.840994	81.1
3538	2023-09-09	77.938004	1987.032679	78.267543	1998.605487	79.034630	2016.822297	80.1
3539	2023-09-10	77.520657	1977.602920	77.893627	1987.637063	78.534824	2005.348232	79.4
3540	2023-09-11	77.227952	1970.399862	77.562204	1978.345153	78.099335	1994.498161	78.9
3541	2023-09-12	76.852786	1963.938361	77.200465	1970.647048	77.734004	1984.626267	78.4
3542	2023-09-13	76.166681	1955.053010	76.749140	1963.130411	77.321384	1975.383737	77.9
3543	2023-09-14	75.782298	1948.391902	76.267255	1955.794425	76.914730	1967.069789	77.4
3544	2023-09-15	75.493726	1941.419794	75.814235	1948.288236	76.507350	1959.467642	77.0
3545	2023-09-16	75.026408	1930.720336	75.434144	1940.177344	76.091642	1951.653878	76.6
3546	2023-09-17	74.674287	1924.784933	75.064807	1932.308354	75.666031	1944.051390	76.2
3547	2023-09-18	74.150603	1917.259441	74.617099	1924.254903	75.215667	1936.271569	75.8
3548	2023-09-19	73.954163	1909.140359	74.259684	1917.061578	74.846914	1928.619461	75.4

	Date	Price_X	Price_Z	X_prom_3	Z_prom_3	X_prom_6	Z_prom_6	X_pr
3549	2023-09-20	73.670149	1900.442208	73.924972	1908.947336	74.494890	1920.627845	75.08
3550	2023-09-21	73.544036	1892.600859	73.722783	1900.727809	74.169941	1912.491356	74.71
3551	2023-09-22	73.218481	1886.032680	73.477556	1893.025249	73.868620	1905.043413	74.39
3552	2023-09-23	73.031215	1880.822652	73.264577	1886.485397	73.594775	1897.716367	74.08
3553	2023-09-24	72.722231	1876.078543	72.990643	1880.977958	73.356713	1890.852883	73.77
3554	2023-09-25	72.161029	1869.749472	72.638159	1875.550222	73.057857	1884.287736	73.45
3555	2023-09-26	71.887000	1865.335810	72.256754	1870.387942	72.760666	1878.436669	73.14
3556	2023-09-27	71.698694	1860.506145	71.915575	1865.197142	72.453109	1873.087550	72.87
3557	2023-09-28	71.324104	1851.761900	71.636599	1859.201285	72.137379	1867.375754	72.58
3558	2023-09-29	71.057615	1847.599313	71.360138	1853.289119	71.808446	1861.838530	72.29

29 rows × 27 columns

In [140…
```
features_final = X_train.columns
```

In [141…
```
# LOWER
X_lower = df_lower_fsct[features_final]

# UPPER
X_upper = df_upper_fsct[features_final]
```

In [142…
```
df_pred_lower = pd.DataFrame({
    "Date": df_lower_fsct["Date"],
    "Equipo1_lower": model1_final.predict(X_lower),
    "Equipo2_lower": model2_final.predict(X_lower)
})

df_pred_upper = pd.DataFrame({
    "Date": df_upper_fsct["Date"],
    "Equipo1_upper": model1_final.predict(X_upper),
    "Equipo2_upper": model2_final.predict(X_upper)
})
```

In [143…

```
df_final_pred = df_pred_F.merge(df_pred_lower, on="Date")
df_final_pred = df_final_pred.merge(df_pred_upper, on="Date")

df_final_pred
```

Out[143...

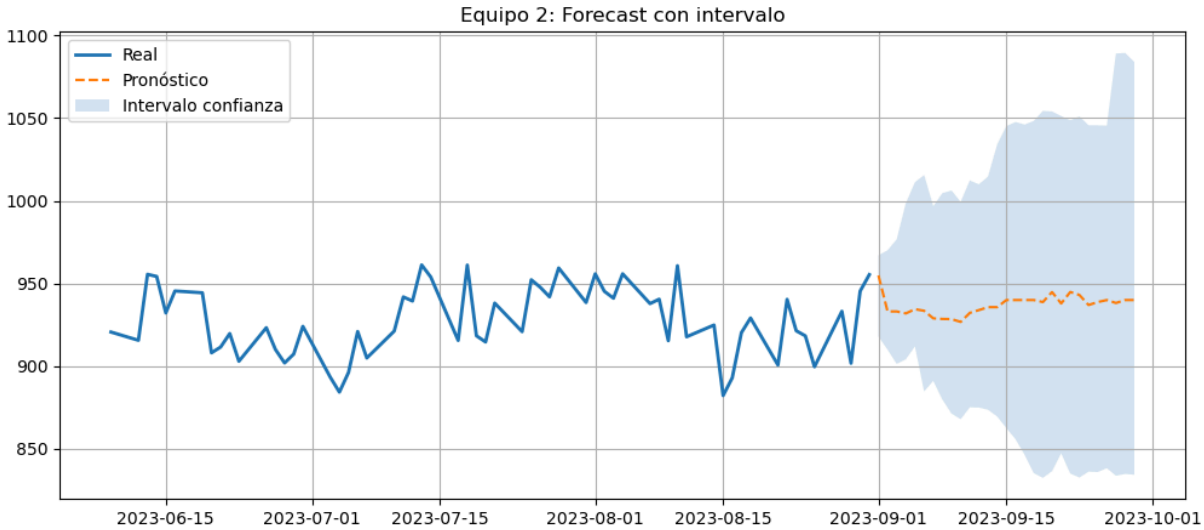
	Date	Price_Equipo1_pred	Price_Equipo2_pred	Equipo1_lower	Equipo2_lower	Equipo1
0	2023-09-01	468.931335	954.877869	466.763184	918.050598	469.
1	2023-09-02	470.946045	933.095398	477.179932	910.037109	485.
2	2023-09-03	472.459564	933.094055	480.101288	901.448181	489.
3	2023-09-04	465.399811	931.915100	472.938568	904.212769	486.
4	2023-09-05	465.087189	934.483276	472.752411	912.087280	494.
5	2023-09-06	491.099792	933.500671	479.904083	884.787598	502.
6	2023-09-07	487.210785	928.940735	472.370758	891.173889	503.
7	2023-09-08	489.326447	928.558594	485.661591	879.899231	500.
8	2023-09-09	483.367249	928.465515	493.405670	871.387573	499.
9	2023-09-10	486.009888	926.782410	497.467010	867.988708	500.
10	2023-09-11	486.009888	932.240417	495.686005	875.161804	509.
11	2023-09-12	487.523407	933.924316	504.038483	875.005615	504.
12	2023-09-13	491.556122	935.700500	498.483795	873.695679	499.
13	2023-09-14	478.723877	935.700500	494.897400	869.600220	495.
14	2023-09-15	479.405243	939.994690	504.778107	862.633667	496.
15	2023-09-16	479.405243	939.994690	482.758026	855.923828	491.
16	2023-09-17	488.196014	939.994690	463.821106	846.508606	503.
17	2023-09-18	485.042542	939.994690	439.340210	835.370789	477.
18	2023-09-19	481.068207	938.776611	432.593231	832.507751	467.

	Date	Price_Equipo1_pred	Price_Equipo2_pred	Equipo1_lower	Equipo2_lower	Equipo1
19	2023-09-20	485.109680	944.824524	444.441040	836.602539	487.
20	2023-09-21	484.832520	937.925964	443.321503	847.433899	476.
21	2023-09-22	481.802521	944.880188	403.029785	835.136230	481.
22	2023-09-23	484.512299	943.048340	454.232483	832.712158	527.
23	2023-09-24	487.134613	937.000427	441.561188	836.287903	554.
24	2023-09-25	487.134613	938.776611	452.049957	836.020813	549.
25	2023-09-26	487.134613	939.994690	452.713715	838.368958	556.
26	2023-09-27	488.937653	938.218506	470.392944	833.794067	563.
27	2023-09-28	491.580292	939.994690	468.690796	834.880920	576.
28	2023-09-29	491.580292	939.994690	467.963470	834.396606	581.

In [144…

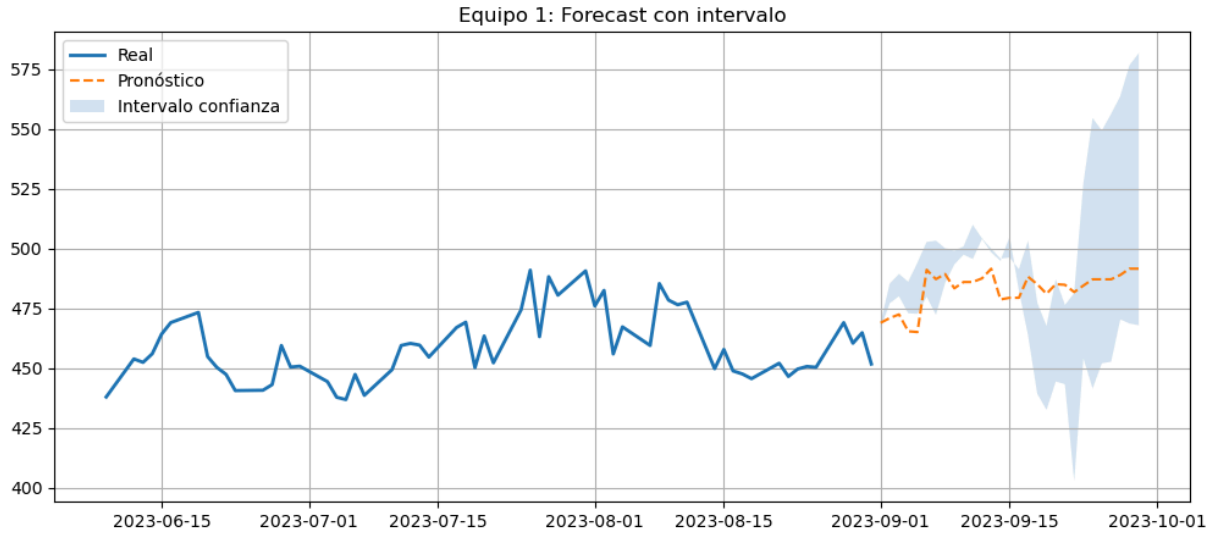
```
import matplotlib.pyplot as plt

plt.figure(figsize=(12,5))

# Real
plt.plot(df.reset_index()["Date"].iloc[-60:],
         df["Price_Equipo2"].iloc[-60:],
         label="Real", linewidth=2)

# Pronóstico medio
plt.plot(df_final_pred["Date"],
         df_final_pred["Price_Equipo2_pred"],
         linestyle="--", label="Pronóstico")

# Intervalo
plt.fill_between(df_final_pred["Date"],
                 df_final_pred["Equipo2_lower"],
                 df_final_pred["Equipo2_upper"],
                 alpha=0.2, label="Intervalo confianza")

plt.title("Equipo 2: Forecast con intervalo")
plt.legend()
plt.grid()
```

```
plt.show()
```



In [145…

```
plt.figure(figsize=(12,5))

plt.plot(df.reset_index()["Date"].iloc[-60:],
         df["Price_Equipo1"].iloc[-60:],
         label="Real", linewidth=2)

plt.plot(df_final_pred["Date"],
         df_final_pred["Price_Equipo1_pred"],
         linestyle="--", label="Pronóstico")

plt.fill_between(df_final_pred["Date"],
                 df_final_pred["Equipo1_lower"],
                 df_final_pred["Equipo1_upper"],
                 alpha=0.2, label="Intervalo confianza")

plt.title("Equipo 1: Forecast con intervalo")
plt.legend()
plt.grid()

plt.show()
```



Suavizamos los intervalos para reducir la incertidumbre

In [146…

```
# Suavizar intervalos
df_final_pred["Equipo2_lower_smooth"] = df_final_pred["Equipo2_lower"].rolling(3).m
df_final_pred["Equipo2_upper_smooth"] = df_final_pred["Equipo2_upper"].rolling(3).m

df_final_pred["Equipo1_lower_smooth"] = df_final_pred["Equipo1_lower"].rolling(3).m
df_final_pred["Equipo1_upper_smooth"] = df_final_pred["Equipo1_upper"].rolling(3).m
```

In [148…

```
df_final_pred
```

Out[148...

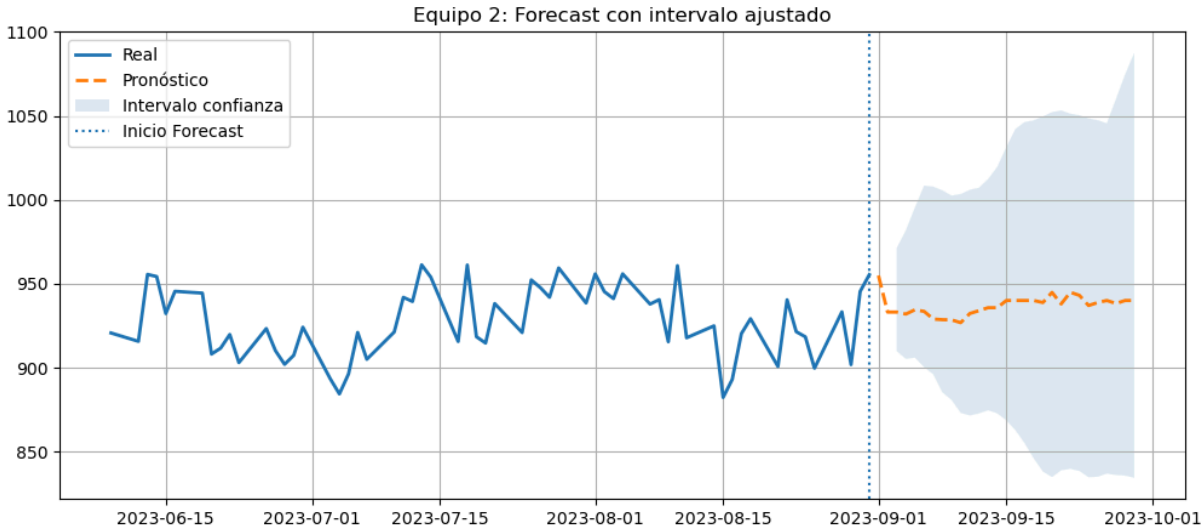
	Date	Price_Equipo1_pred	Price_Equipo2_pred	Equipo1_lower	Equipo2_lower	Equipo1
0	2023-09-01	468.931335	954.877869	466.763184	918.050598	469.
1	2023-09-02	470.946045	933.095398	477.179932	910.037109	485.
2	2023-09-03	472.459564	933.094055	480.101288	901.448181	489.
3	2023-09-04	465.399811	931.915100	472.938568	904.212769	486.
4	2023-09-05	465.087189	934.483276	472.752411	912.087280	494.
5	2023-09-06	491.099792	933.500671	479.904083	884.787598	502.
6	2023-09-07	487.210785	928.940735	472.370758	891.173889	503.
7	2023-09-08	489.326447	928.558594	485.661591	879.899231	500.
8	2023-09-09	483.367249	928.465515	493.405670	871.387573	499.
9	2023-09-10	486.009888	926.782410	497.467010	867.988708	500.
10	2023-09-11	486.009888	932.240417	495.686005	875.161804	509.
11	2023-09-12	487.523407	933.924316	504.038483	875.005615	504.
12	2023-09-13	491.556122	935.700500	498.483795	873.695679	499.
13	2023-09-14	478.723877	935.700500	494.897400	869.600220	495.
14	2023-09-15	479.405243	939.994690	504.778107	862.633667	496.
15	2023-09-16	479.405243	939.994690	482.758026	855.923828	491.
16	2023-09-17	488.196014	939.994690	463.821106	846.508606	503.
17	2023-09-18	485.042542	939.994690	439.340210	835.370789	477.
18	2023-09-19	481.068207	938.776611	432.593231	832.507751	467.

	Date	Price_Equipo1_pred	Price_Equipo2_pred	Equipo1_lower	Equipo2_lower	Equipo1
19	2023-09-20	485.109680	944.824524	444.441040	836.602539	487.
20	2023-09-21	484.832520	937.925964	443.321503	847.433899	476.
21	2023-09-22	481.802521	944.880188	403.029785	835.136230	481.
22	2023-09-23	484.512299	943.048340	454.232483	832.712158	527.
23	2023-09-24	487.134613	937.000427	441.561188	836.287903	554.
24	2023-09-25	487.134613	938.776611	452.049957	836.020813	549.
25	2023-09-26	487.134613	939.994690	452.713715	838.368958	556.
26	2023-09-27	488.937653	938.218506	470.392944	833.794067	563.
27	2023-09-28	491.580292	939.994690	468.690796	834.880920	576.
28	2023-09-29	491.580292	939.994690	467.963470	834.396606	581.

In [149…

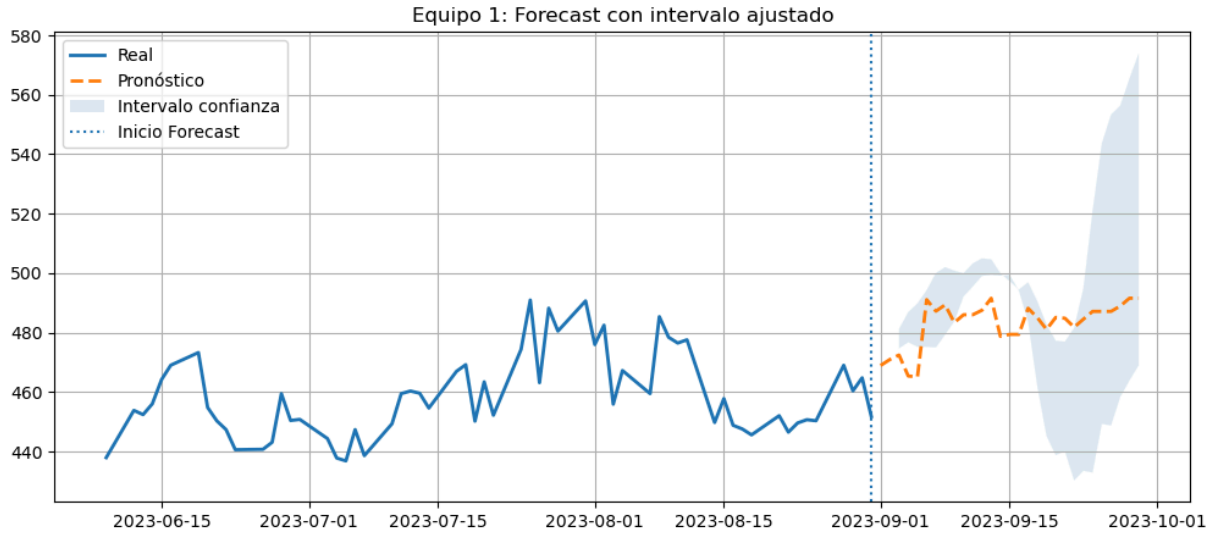
```
plt.figure(figsize=(12,5))

# Real
plt.plot(df.reset_index()["Date"].iloc[-60:],
         df["Price_Equipo2"].iloc[-60:],
         label="Real", linewidth=2)

# Forecast
plt.plot(df_final_pred["Date"],
         df_final_pred["Price_Equipo2_pred"],
         linestyle="--", linewidth=2, label="Pronóstico")

# Intervalo limpio
plt.fill_between(df_final_pred["Date"],
                 df_final_pred["Equipo2_lower_smooth"],
                 df_final_pred["Equipo2_upper_smooth"],
                 alpha=0.15, label="Intervalo confianza")

plt.axvline(x=df.reset_index()["Date"].iloc[-1], linestyle=":", label="Inicio Forec

plt.title("Equipo 2: Forecast con intervalo ajustado")
plt.legend()
plt.grid()
```

```
plt.show()
```



In [150…

```
plt.figure(figsize=(12,5))

plt.plot(df.reset_index()["Date"].iloc[-60:],
         df["Price_Equipo1"].iloc[-60:],
         label="Real", linewidth=2)

plt.plot(df_final_pred["Date"],
         df_final_pred["Price_Equipo1_pred"],
         linestyle="--", linewidth=2, label="Pronóstico")

plt.fill_between(df_final_pred["Date"],
                 df_final_pred["Equipo1_lower_smooth"],
                 df_final_pred["Equipo1_upper_smooth"],
                 alpha=0.15, label="Intervalo confianza")

plt.axvline(x=df.reset_index()["Date"].iloc[-1], linestyle=":", label="Inicio Forec

plt.title("Equipo 1: Forecast con intervalo ajustado")
plt.legend()
plt.grid()

plt.show()
```



In [151…

```
df_final_pred[["Date","Price_Equipo1_pred","Equipo1_upper","Equipo1_lower"]]
```

Out[151…

	Date	Price_Equipo1_pred	Equipo1_upper	Equipo1_lower
0	2023-09-01	468.931335	469.002014	466.763184
1	2023-09-02	470.946045	485.315277	477.179932
2	2023-09-03	472.459564	489.327362	480.101288
3	2023-09-04	465.399811	486.072571	472.938568
4	2023-09-05	465.087189	494.035797	472.752411
5	2023-09-06	491.099792	502.713715	479.904083
6	2023-09-07	487.210785	503.285919	472.370758
7	2023-09-08	489.326447	500.104187	485.661591
8	2023-09-09	483.367249	499.077271	493.405670
9	2023-09-10	486.009888	500.813599	497.467010
10	2023-09-11	486.009888	509.900757	495.686005
11	2023-09-12	487.523407	504.195099	504.038483
12	2023-09-13	491.556122	499.907684	498.483795
13	2023-09-14	478.723877	495.792938	494.897400
14	2023-09-15	479.405243	496.300934	504.778107
15	2023-09-16	479.405243	491.337921	482.758026
16	2023-09-17	488.196014	503.139862	463.821106
17	2023-09-18	485.042542	477.245392	439.340210
18	2023-09-19	481.068207	467.542816	432.593231
19	2023-09-20	485.109680	487.039581	444.441040
20	2023-09-21	484.832520	476.349030	443.321503
21	2023-09-22	481.802521	481.319855	403.029785
22	2023-09-23	484.512299	527.063293	454.232483
23	2023-09-24	487.134613	554.524292	441.561188
24	2023-09-25	487.134613	549.304749	452.049957
25	2023-09-26	487.134613	556.178162	452.713715
26	2023-09-27	488.937653	563.614441	470.392944
27	2023-09-28	491.580292	576.590698	468.690796
28	2023-09-29	491.580292	581.620728	467.963470

- Se realizó el pronóstico de los precios de los equipos 1 y 2 utilizando un modelo XGBoost. Previamente, se proyectaron las variables exógenas para alimentar el modelo, permitiendo generar no solo una estimación puntual, sino también intervalos de confianza (upper y lower), con el fin de brindar al usuario un rango probable en el que se puede ubicar el precio final.
- El horizonte de predicción se estableció en 30 días, considerando que, a medida que aumenta el periodo de pronóstico, también lo hace la incertidumbre. Aunque el modelo tiene la capacidad de proyectar a plazos más largos, desde una perspectiva estadística es más confiable trabajar con horizontes más acotados.
- Adicionalmente, se desarrolló un agente capaz de generar predicciones para distintos horizontes de tiempo según se requiera, acompañado de su respectivo análisis e interpretación. Este agente sigue la lógica del modelo propuesto e incorpora, a nivel contextual, factores macroeconómicos y eventos relevantes (como noticias), con el objetivo de enriquecer la lectura de los resultados.